

LE COLLÉ (INFORMATIQUE)

Résumé complet

Table des matières

1 Codage des nombres entiers	5	7 Tuples, ensembles et dictionnaires	15
1.1 Les bases de numération	5	7.1 Les tuples (<code>tuple</code>)	15
1.2 Conversion de base 10 vers une base b . . .	5	7.1.1 Propriétés	15
1.3 Conversion de la base 2 vers les bases 2^k . .	5	7.1.2 Opérations	15
1.4 Codage des entiers naturels (non signés) . .	5	7.1.3 Déballage (<code>unpacking</code>)	15
1.5 Codage des entiers relatifs	5	7.2 Les ensembles (<code>set</code>)	16
1.5.1 Signe et valeur absolue	5	7.2.1 Opérations de base	16
1.5.2 Complément à 2	6	7.2.2 Opérations ensemblistes	16
1.6 Dépassement de capacité	6	7.3 Les dictionnaires (<code>dict</code>)	16
2 Codage des nombres réels	6	7.3.1 Création	16
2.1 Représentation en virgule fixe	6	7.3.2 Accès et modification	16
2.2 Conversion base 10 vers base 2	6	7.3.3 Suppression	17
2.3 Représentation en virgule flottante – Norme IEEE 754	7	7.3.4 Parcours	17
2.3.1 Formats courants	7	7.3.5 Test d'appartenance	17
2.3.2 Exemple : codage de $-54,25$ en simple précision	7	7.3.6 Complexité des dictionnaires	17
2.4 Cas particuliers (dénormalisés, infinis, NaN)	7	8 Les fichiers	17
2.5 Erreurs d'arrondi et précautions	7	8.1 Ouverture et fermeture d'un fichier	17
3 Introduction au langage Python	7	8.2 Lecture d'un fichier texte	17
3.1 Variables et types	7	8.3 Écriture dans un fichier texte	18
3.2 Opérations arithmétiques	8	8.4 Gestion des chemins	18
3.3 Entrées / sorties	8	8.5 Le module <code>os</code>	18
3.4 Structures conditionnelles	8	9 Bibliothèques mathématiques	18
3.5 Boucles	8	9.1 Module <code>math</code>	18
3.6 Fonctions	8	9.2 Module <code>numpy</code>	18
3.7 Portée des variables	8	9.2.1 Création de tableaux	19
3.8 Récursivité	9	9.2.2 Propriétés et accès	19
3.9 Modules	9	9.2.3 Opérations vectorisées	19
4 Les chaînes de caractères	9	9.2.4 Algèbre linéaire avec <code>numpy.linalg</code>	19
4.1 Définition	9	9.3 Module <code>random</code>	19
4.2 Accès aux caractères – indices	9	9.4 Module <code>matplotlib</code>	20
4.3 Fonctions et opérations sur les chaînes . . .	9	9.5 Module <code>scipy</code> (complément)	20
4.4 Méthodes courantes	10	10 Les arbres binaires	20
4.5 Parcours d'une chaîne	10	10.1 Définitions	20
4.6 Codage des caractères	10	10.2 Représentation en Python	21
5 Les listes	10	10.3 Parcours d'un arbre binaire	21
5.1 Définition et création	10	10.3.1 Parcours préfixe (racine, gauche, droit)	21
5.2 Accès et modification – indices	11	10.3.2 Parcours infixe (gauche, racine, droit)	21
5.3 Méthodes courantes	11	10.3.3 Parcours postfixe (gauche, droit, racine)	21
5.4 Parcours	11	10.3.4 Parcours en largeur (BFS)	21
5.5 Listes en compréhension	12	10.4 Arbres binaires de recherche (ABR)	22
5.6 Matrices (listes de listes)	12	10.4.1 Recherche dans un ABR	22
5.7 Copies : référence vs copie profonde	12	10.4.2 Insertion dans un ABR	22
6 Algorithmes de tri et de recherche	12	10.5 Tas (heap)	22
6.1 Recherche séquentielle (linéaire)	12	10.5.1 Représentation par tableau	22
6.2 Recherche dichotomique	12	10.5.2 Tamisage (heapify)	22
6.3 Recherche d'un mot dans une chaîne (naïve) . . .	13	10.5.3 Construction d'un tas	23
6.4 Tri par sélection	13	10.5.4 Tri par tas (Heap sort)	23
6.5 Tri par insertion	13	10.5.5 File de priorité avec tas	23
6.6 Tri à bulles	14		
6.7 Tri rapide (Quick sort)	14		
6.8 Tri fusion (Merge sort)	14		
6.9 Tri par tas (Heap sort)	15		
6.10 Bilan des complexités	15		
6.11 Tri en Python : <code>sort()</code> et <code>sorted()</code>	15		

11 Les graphes	23	15 Complexité algorithmique	34
11.1 Définitions générales	23	15.1 Notations asymptotiques	34
11.2 Vocabulaire	23	15.2 Règles de calcul de complexité	35
11.3 Représentations en Python	24	15.3 Exemples de calcul	35
11.3.1 Matrice d'adjacence	24	15.4 Ordres de grandeur usuels	36
11.3.2 Liste d'adjacence	24	15.5 Théorème maître	36
11.4 Parcours de graphes	24	16 Algorithmes gloutons	36
11.4.1 Parcours en largeur (BFS)	24	16.1 Principe	36
11.4.2 Parcours en profondeur (DFS)	24	16.2 Caractéristiques	36
11.5 Détection de cycles	25	16.3 Exemple : problème de planification (interval scheduling)	36
11.5.1 Graphe non orienté	25	16.4 Exemple : rendu de monnaie glouton (systèmes canoniques)	36
11.5.2 Graphe orienté	25	16.5 Exemple : sac à dos fractionnaire	37
11.6 Plus courts chemins – Dijkstra	25	17 Bases de données relationnelles et SQL	37
11.7 Plus courts chemins tous couples – Floyd-Warshall	25	17.1 Concepts fondamentaux	37
12 Programmation dynamique	25	17.2 Modèle entité-association (conceptuel)	37
12.1 Principes fondamentaux	26	17.3 Modèle relationnel (logique)	37
12.2 Exemple : suite de Fibonacci	26	17.4 SQL : Structured Query Language	38
12.3 Problème du rendu de monnaie	26	17.4.1 Création de tables	38
12.4 Problème du sac à dos 0/1	27	17.4.2 Insertion, mise à jour, suppression	38
12.5 Plus longue sous-séquence commune (LCS)	27	17.4.3 Interrogation : SELECT	38
12.6 Distance de Levenshtein	27	17.5 Ordre logique d'exécution d'une requête	39
12.6.1 Relation de récurrence	28	17.6 Utilisation en Python avec sqlite3	39
12.6.2 Tabulation (Bottom-Up)	28	18 Apprentissage automatique	40
12.7 Autres exemples classiques	28	18.1 Introduction	40
13 Traitement des images	28	18.2 Apprentissage supervisé : k plus proches voisins (k-NN)	40
13.1 Introduction aux images numériques	28	18.2.1 Principe	40
13.2 Chargement et affichage d'une image avec Matplotlib	28	18.2.2 Distance euclidienne	40
13.3 Opérations de base sur les images	29	18.2.3 Algorithme k-NN complet	41
13.3.1 Négatif	29	18.2.4 Choix de k et normalisation	41
13.3.2 Inversion verticale	29	18.3 Apprentissage non supervisé : k-moyennes (k-means)	41
13.3.3 Miroir horizontal	29	18.3.1 Algorithme	41
13.3.4 Rotation à 90°	29	18.3.2 Choix du nombre de clusters	41
13.3.5 Extraction d'une moitié	30	18.4 Évaluation des performances en classification	42
13.4 Manipulation des couleurs	30	18.4.1 Matrice de confusion	42
13.4.1 Conversion en niveaux de gris	30	19 Théorie des jeux	42
13.4.2 Conversion en noir et blanc (seuillage)	30	19.1 Définitions	42
13.4.3 Extraction des canaux RVB	30	19.2 Attracteurs	42
13.5 Redimensionnement d'images	30	19.3 Jeu de Nim et nombre de Grundy	43
13.5.1 Agrandissement par répétition	30	19.4 Algorithme Min-Max (pour jeux partisans)	43
13.5.2 Réduction par moyennage	31	19.5 Élagage alpha-bêta	43
13.6 Superposition d'images (mélange)	31	19.6 Heuristique	44
13.7 Filtrage par convolution	32	20 Algorithmes de graphes avancés	44
14 Méthodes numériques	32	20.1 Algorithme de Bellman-Ford (plus courts chemins avec poids négatifs)	44
14.1 Calculs approchés d'intégrales	32	20.2 Algorithme de Floyd-Warshall (tous les couples)	44
14.1.1 Méthode des rectangles	32	20.3 Algorithme de Johnson (tous les couples, poids négatifs)	45
14.1.2 Méthode des trapèzes	32	20.4 Arbres couvrants de poids minimal	45
14.1.3 Méthode de Simpson	33	20.4.1 Algorithme de Prim	45
14.2 Résolution approchée de $f(x) = 0$	33	20.4.2 Algorithme de Kruskal	45
14.2.1 Méthode de dichotomie	33	20.5 Algorithme A* (recherche de chemin avec heuristique)	46
14.2.2 Méthode de Newton-Raphson	33	20.6 Ordre topologique (graphe orienté acyclique)	47
14.3 Équations différentielles ordinaires	34	20.7 Composantes fortement connexes (Kosaraju)	47
14.3.1 Méthode d'Euler explicite	34		
14.3.2 Méthode de Runge-Kutta d'ordre 4 (RK4)	34		
14.3.3 Utilisation de <code>scipy.integrate.odeint</code>	34		

21 Tables de hachage et recherche de motif	48	22.5 Tests unitaires	51
21.1 Tables de hachage	48	22.6 Contrôle de version	51
21.1.1 Fonction de hachage	48	23 Preuves de correction et terminaison	51
21.1.2 Gestion des collisions	48	23.1 Terminaison d'une boucle	51
21.2 Recherche de motif dans un texte	49	23.2 Correction d'une boucle : invariant	51
21.2.1 Algorithme naïf	49	23.3 Correction récursive	52
21.2.2 Algorithme de Rabin-Karp	49	23.4 Complexité et preuve	52
21.2.3 Algorithme de Knuth-Morris-Pratt (KMP)	49	24 Récursivité avancée	52
22 Bonnes pratiques de programmation	50	24.1 Récursivité terminale	52
22.1 Documentation et lisibilité	50	24.2 Mémoïsation (caching)	52
22.2 Assertions et programmation défensive	50	24.3 Récursivité mutuelle	52
22.3 Gestion des exceptions	50	24.4 Récursivité sur des structures arborescentes	53
22.4 Modularité et réutilisabilité	51	24.5 Dépassement de pile et profondeur limite	53

1 Codage des nombres entiers

1.1 Les bases de numération

Un nombre entier peut être représenté dans une base $b \geq 2$ par une suite de chiffres $(a_p a_{p-1} \dots a_0)_b$ avec $0 \leq a_i \leq b - 1$. Sa valeur en base 10 est :

$$(a_p a_{p-1} \dots a_0)_b = \sum_{i=0}^p a_i b^i.$$

Bases usuelles :

- Binaire (base 2) : chiffres 0,1.
- Octal (base 8) : chiffres 0..7.
- Hexadécimal (base 16) : chiffres 0..9, A..F.

En Python, les nombres en binaire s'écrivent avec le préfixe `0b`, en hexadécimal avec `0x`. La fonction `bin(x)` convertit en binaire, `hex(x)` en hexadécimal.

1.2 Conversion de base 10 vers une base b

Méthode des divisions successives :

1. Diviser le nombre par b , le reste est le chiffre de poids faible.
2. Recommencer avec le quotient jusqu'à obtenir 0.
3. Les restes lus de bas en haut donnent l'écriture en base b .

Exemple : 42_{10} en binaire

$$\begin{aligned} 42 \div 2 &= 21 \text{ reste } 0 \\ 21 \div 2 &= 10 \text{ reste } 1 \\ 10 \div 2 &= 5 \text{ reste } 0 \\ 5 \div 2 &= 2 \text{ reste } 1 \\ 2 \div 2 &= 1 \text{ reste } 0 \\ 1 \div 2 &= 0 \text{ reste } 1 \end{aligned}$$

Lecture de bas en haut : 101010_2 .

1.3 Conversion de la base 2 vers les bases 2^k

On regroupe les bits par paquets de k (de droite à gauche) et on convertit chaque groupe en le chiffre correspondant. Exemples :

- Base 8 : groupes de 3 bits. $1010011101_2 = 001\ 010\ 011\ 101 = 1235_8$.
- Base 16 : groupes de 4 bits. $1010011101_2 = 0010\ 1001\ 1101 = 29D_{16}$.

1.4 Codage des entiers naturels (non signés)

Sur n bits, un entier non signé prend des valeurs de 0 à $2^n - 1$. Codage : conversion binaire classique, compléter à gauche par des zéros.

Nombre de bits	Minimum	Maximum
8	0	255
16	0	65 535
32	0	4 294 967 295

1.5 Codage des entiers relatifs

1.5.1 Signe et valeur absolue

Le bit de poids fort indique le signe (0 pour positif, 1 pour négatif), les $n - 1$ bits restants codent la valeur absolue. Problème : deux représentations pour 0, addition complexe.

1.5.2 Complément à 2

- Les entiers positifs sont codés comme en binaire.
- Pour un entier négatif $-x$ ($x > 0$), on code x en binaire, on inverse tous les bits (complément à 1), puis on ajoute 1.
- Le bit de poids fort est toujours 1 pour les négatifs, 0 pour les positifs.
- Intervalle de représentation sur n bits : $[-2^{n-1}, 2^{n-1} - 1]$.
- Avantage : l'addition binaire fonctionne sans traitement spécial.

Exemple : codage de -24 sur 8 bits

$$\begin{aligned} 24_{10} &= 00011000_2 \\ \text{complément à 1} &= 11100111_2 \\ \text{ajouter 1} &= 11101000_2 \end{aligned}$$

Donc $-24_{10} = 11101000_{\text{ca2}}$.

1.6 Dépassement de capacité

En complément à 2, un dépassement (overflow) se produit lorsque le résultat d'une opération sort de l'intervalle représentable. Le processeur le signale par un drapeau. En Python, les entiers sont de taille arbitraire, donc pas de dépassement.

2 Codage des nombres réels**2.1 Représentation en virgule fixe**

En base b , un nombre réel x s'écrit

$$(x)_b = a_n a_{n-1} \dots a_0, c_1 c_2 \dots c_m$$

Il se convertit en décimal par :

$$(x)_{10} = \sum_{i=0}^n a_i b^i + \sum_{j=1}^m c_j b^{-j}$$

2.2 Conversion base 10 vers base 2

Pour convertir un nombre réel en binaire, on procède comme suit :

- Partie entière : divisions successives par 2.
- Partie fractionnaire : on multiplie la partie fractionnaire par 2, on note la partie entière obtenue, on recommence avec la nouvelle partie fractionnaire jusqu'à obtenir 0 ou la précision souhaitée.

Exemple : conversion de $42,375_{10}$ en binaire

- Partie entière : $42_{10} = 101010_2$
- Partie fractionnaire :

$$\begin{aligned} 0,375 \times 2 &= 0,75 & (0) \\ 0,75 \times 2 &= 1,5 & (1) \\ 0,5 \times 2 &= 1,0 & (1) \end{aligned}$$

$$\text{donc } 0,375_{10} = 0,011_2$$

Résultat : $42,375_{10} = 101010,011_2$

2.3 Représentation en virgule flottante – Norme IEEE 754

Un nombre x s'écrit sous forme normalisée :

$$x = (-1)^s \times 1, m \times 2^p$$

où s est le signe (0 pour positif, 1 pour négatif), m la mantisse (partie fractionnaire), p l'exposant réel.

2.3.1 Formats courants

- **Simple précision (32 bits)** : 1 bit de signe, 8 bits d'exposant, 23 bits de mantisse.
- **Double précision (64 bits)** : 1 bit de signe, 11 bits d'exposant, 52 bits de mantisse.

L'exposant est stocké **décalé** : $e = p + \text{décalage}$ avec décalage 127 pour la simple précision et 1023 pour la double précision.

2.3.2 Exemple : codage de $-54,25$ en simple précision

- Signe $s = 1$.
- Valeur absolue $54,25_{10} = 110110,01_2 = 1,1011001_2 \times 2^5$.
- Exposant stocké : $5 + 127 = 132 = 10000100_2$.
- Mantisse : les bits après la virgule (23 bits) : 1011001 0000000000000000.
- Codage final : 1 10000100 1011001 0000000000000000.

2.4 Cas particuliers (dénormalisés, infinis, NaN)

- **Zéro** : $E = 0, M = 0$ (deux zéros : $+0$ et -0).
- **Infini** : $E = 2^{n_E} - 1, M = 0$ (signe $\pm$).
- **NaN (Not a Number)** : $E = 2^{n_E} - 1, M \neq 0$.
- **Dénormalisés** : $E = 0, M \neq 0$, utilisés pour représenter des nombres proches de zéro.

2.5 Erreurs d'arrondi et précautions

Précautions avec les flottants :

- Ne jamais tester l'égalité exacte (préférer une tolérance).
- Additionner les petits avant les grands.
- Éviter les soustractions de nombres très proches (perte de précision).
- Utiliser `math.isclose(a,b)` pour comparer des flottants.

3 Introduction au langage Python

3.1 Variables et types

En Python, une variable est créée par affectation. Le type est dynamique (déduit automatiquement). Types simples :

- `int` : entiers (taille arbitraire)
- `float` : flottants double précision
- `bool` : `True` ou `False`
- `str` : chaînes de caractères (immuables)
- `None` : absence de valeur

Conversion de type : `int(x)`, `float(x)`, `str(x)`.

3.2 Opérations arithmétiques

```

1 + - * / # division flottante
2 // # division entière (troncature vers -inf)
3 % # modulo
4 ** # puissance

```

Priorité des opérateurs : parenthèses, puis **, puis *, /, //, %, puis +, -. En cas d'égalité, associativité gauche (sauf ** qui est droite).

3.3 Entrées / sorties

```

1 x = input("Entrez un nombre : ") # retourne une chaîne
2 print("La valeur est", x)
3 print(f"La valeur est {x}")      # f-string (formatage)

```

Paramètres de print : `sep` (séparateur, défaut espace), `end` (fin de ligne, défaut `'\n'`).

3.4 Structures conditionnelles

```

1 if condition:
2     bloc
3 elif condition:
4     bloc
5 else:
6     bloc

```

Opérateurs de comparaison : `==` `!=` `<` `>` `<=` `>=`

Opérateurs logiques : `and` `or` `not` (évaluation paresseuse). Toute valeur peut être interprétée comme booléenne : `False` pour 0, 0.0, None, "", [], (), {}, `set()` ; `True` pour les autres.

3.5 Boucles

```

1 for i in range(debut, fin, pas):
2     bloc
3 while condition:
4     bloc

```

`break` : sort de la boucle ; `continue` : passe à l'itération suivante. `range(n)` génère les entiers de 0 à n-1.

3.6 Fonctions

```

1 def nom_fonction(param1, param2=defaut):
2     """docstring"""
3     instructions
4     return valeur # optionnel

```

Une fonction sans `return` retourne `None`. Les paramètres avec valeurs par défaut sont optionnels.

3.7 Portée des variables

- **Locale** : créée dans une fonction, n'existe qu'à l'intérieur.
- **Globale** : définie en dehors, accessible partout (sauf si masquée par une locale).
- Pour modifier une variable globale dans une fonction, utiliser `global var`.

3.8 Récursivité

Une fonction récursive s'appelle elle-même. Doit comporter un cas de base et des appels qui se rapprochent du cas de base.

```
1 def factorielle(n):
2     if n <= 1:
3         return 1
4     return n * factorielle(n-1)
```

La profondeur de récursion est limitée (par défaut 1000). On peut la modifier avec `sys.setrecursionlimit()`.

3.9 Modules

Import d'un module entier : `import math` puis `math.sqrt(2)`. Import partiel : `from math import sqrt, pi`. Import avec alias : `import numpy as np`.

4 Les chaînes de caractères

4.1 Définition

Une chaîne de caractères (type `str`) est une séquence ordonnée et **immuable** de caractères. On la délimite par des guillemets simples (') ou doubles ("), ou par des triples guillemets (""") pour les chaînes multi-lignes.

```
1 ch1 = "Bonjour"
2 ch2 = 'Hello'
3 ch3 = """Ceci est
4 une chaîne
5 multi-lignes"""
```

4.2 Accès aux caractères – indices

Les caractères sont indexés à partir de 0. On peut utiliser des indices positifs (de gauche à droite) ou négatifs (de droite à gauche, -1 désigne le dernier caractère).

```
1 s = "Python"
2 print(s[0])    # 'P'
3 print(s[-1])   # 'n'
4 print(s[1:4])  # 'yth' (tranche : du 1 inclus au 4 exclus)
```

Une tentative de modification par affectation provoque une erreur (immuabilité) :

```
1 s[0] = 'J'    # TypeError
```

4.3 Fonctions et opérations sur les chaînes

- `len(s)` : longueur de la chaîne.
- `ord(c)` : code ASCII (ou Unicode) du caractère.
- `chr(n)` : caractère correspondant au code.
- `s1 + s2` : concaténation.
- `s * n` : répétition.
- `c in s` : test d'appartenance.

```
1 len("Python")    # 6
2 ord('a')         # 97
3 chr(65)          # 'A'
```

```

4 "Hello" + " " + "World" # "Hello World"
5 "Ha" * 3 # "HaHaHa"
6 'P' in "Python" # True

```

4.4 Méthodes courantes

Les méthodes ne modifient pas la chaîne originale (elles retournent une nouvelle chaîne).

- `s.upper()` / `s.lower()` : conversion en majuscules/minuscules.
- `s.find(sous)` : retourne l'indice de la première occurrence de `sous`, ou -1 si absent.
- `s.index(sous)` : comme `find` mais lève une exception si absent.
- `s.split(sep)` : découpe la chaîne selon le séparateur `sep` et retourne une liste de morceaux (par défaut les espaces).
- `s.join(liste)` : inverse de `split`, concatène les éléments de la liste avec `s` comme séparateur.
- `s.count(sous)` : nombre d'occurrences de `sous`.
- `s.replace(old, new)` : remplace toutes les occurrences de `old` par `new`.
- `s.strip()` : supprime les espaces au début et à la fin.

```

1 ch = "Bonjour tout le monde"
2 ch.upper() # "BONJOUR TOUT LE MONDE"
3 ch.find("jour") # 3
4 ch.split(" ") # ['Bonjour', 'tout', 'le', 'monde']
5 "-".join(['a', 'b', 'c']) # "a-b-c"
6 ch.count('o') # 4
7 ch.replace('o', 'a') # "Banjaur taut le mande"
8 " texte ".strip() # "texte"

```

4.5 Parcours d'une chaîne

On peut parcourir par caractère :

```

1 for c in ch:
2     print(c)

```

Ou par indice :

```

1 for i in range(len(ch)):
2     print(ch[i])

```

4.6 Codage des caractères

La table ASCII code chaque caractère sur 7 bits (0-127). Les codes 0-31 sont des caractères de contrôle. Exemples :

- 'A' → 65 (0x41)
- 'a' → 97 (0x61)
- '0' → 48 (0x30)

Unicode (UTF-8) est un sur-ensemble compatible ASCII.

5 Les listes

5.1 Définition et création

Une liste (type `list`) est une séquence ordonnée et **muable** d'éléments (qui peuvent être de types différents). On la définit avec des crochets.

```

1 L = [1, 2, 3, 4]
2 mixte = [1, "deux", 3.0, True]
3 vide = []

```

Création par conversion : `list(iterable)`.

```

1 L = list(range(5))    # [0,1,2,3,4]

```

5.2 Accès et modification – indices

Comme pour les chaînes, les indices commencent à 0. On peut accéder en lecture et en écriture.

```

1 L = [10, 20, 30]
2 L[1] = 99
3 print(L)    # [10, 99, 30]
4 print(L[-1]) # 30

```

Les tranches (slicing) retournent une **nouvelle** liste.

```

1 L[1:3]    # [99, 30]
2 L[:2]     # [10, 99]

```

5.3 Méthodes courantes

- `L.append(x)` : ajoute `x` en fin de liste ($O(1)$ amorti).
- `L.pop(i)` : retire et retourne l'élément à l'indice `i` (par défaut le dernier) ($O(1)$ pour fin, $O(n)$ pour début).
- `L.insert(i, x)` : insère `x` à l'indice `i` ($O(n)$).
- `L.remove(x)` : retire la première occurrence de `x` (erreur si absent).
- `L.index(x)` : retourne l'indice de la première occurrence de `x`.
- `L.count(x)` : nombre d'occurrences de `x`.
- `L.sort()` : trie la liste en place (ordre croissant).
- `L.reverse()` : inverse la liste en place.
- `L.copy()` : retourne une copie superficielle de la liste.
- `L.extend(L2)` : ajoute tous les éléments de `L2` à la fin de `L`.

```

1 L = [3,1,2]
2 L.append(4)           # [3,1,2,4]
3 L.pop()              # 4, L devient [3,1,2]
4 L.insert(1,99)       # [3,99,1,2]
5 L.remove(99)         # [3,1,2]
6 L.sort()             # [1,2,3]
7 L.reverse()          # [3,2,1]

```

Attention : `L.sort()` modifie la liste et retourne `None`. Pour obtenir une liste triée sans modifier l'originale, utiliser `sorted(L)`.

5.4 Parcours

```

1 for x in L:
2     print(x)

```

Si on a besoin des indices, on utilise `range(len(L))` ou `enumerate` :

```

1 for i, val in enumerate(L):
2     print(i, val)

```

5.5 Listes en compréhension

Syntaxe : `[expression for variable in iterable if condition]`. Permet de créer des listes de manière concise.

```
1 carres = [x**2 for x in range(10)]          # [0,1,4,9,16,25,36,49,64,81]
2 pairs = [x for x in range(20) if x%2 == 0] # [0,2,4,...,18]
```

5.6 Matrices (listes de listes)

Une matrice peut être représentée par une liste de lignes, chaque ligne étant une liste.

```
1 M = [[1,2,3],
2      [4,5,6],
3      [7,8,9]]
4 print(M[1][2])    # 6
```

Pour parcourir tous les éléments :

```
1 for i in range(len(M)):
2     for j in range(len(M[i])):
3         print(M[i][j], end=' ')
4     print()
```

Création d'une matrice remplie de zéros (attention aux références) :

```
1 M = [[0]*3 for _ in range(3)]    # [[0,0,0],[0,0,0],[0,0,0]]
2 # NE PAS faire : [[0]*3]*3 car cela crée des références à la même ligne
```

5.7 Copies : référence vs copie profonde

En Python, une variable est une **référence** vers un objet. L'affectation `L2 = L` ne copie pas la liste, mais copie la référence. Pour une copie superficielle : `L2 = L[:]` ou `L2 = L.copy()`. Pour une copie profonde (utile pour les listes imbriquées) : `import copy; L2 = copy.deepcopy(L)`.

6 Algorithmes de tri et de recherche

6.1 Recherche séquentielle (linéaire)

Parcourt la liste jusqu'à trouver l'élément. Complexité : $O(n)$.

```
1 def recherche_sequentielle(L, x):
2     for i in range(len(L)):
3         if L[i] == x:
4             return i
5     return -1
```

6.2 Recherche dichotomique

Nécessite une liste triée. Complexité : $O(\log n)$.

```
1 def recherche_dichotomique(L, x):
2     g, d = 0, len(L)-1
3     while g <= d:
4         m = (g+d)//2
5         if L[m] == x:
6             return m
7         elif L[m] < x:
```

```

8         g = m+1
9     else:
10        d = m-1
11    return -1

```

Version récursive :

```

1 def dichot_rec(L, x, g, d):
2     if g > d:
3         return -1
4     m = (g+d)//2
5     if L[m] == x:
6         return m
7     elif L[m] < x:
8         return dichot_rec(L, x, m+1, d)
9     else:
10        return dichot_rec(L, x, g, m-1)

```

6.3 Recherche d'un mot dans une chaîne (naïve)

On teste toutes les positions de départ. Complexité $O(n \cdot m)$ (n longueur du texte, m longueur du mot).

```

1 def recherche_mot(texte, mot):
2     n, m = len(texte), len(mot)
3     for i in range(n - m + 1):
4         j = 0
5         while j < m and texte[i+j] == mot[j]:
6             j += 1
7         if j == m:
8             return i
9     return -1

```

6.4 Tri par sélection

Principe : à chaque étape, on cherche le minimum du reste du tableau et on l'échange avec l'élément courant. Complexité : $O(n^2)$ dans tous les cas. Non stable.

```

1 def tri_selection(L):
2     n = len(L)
3     for i in range(n-1):
4         i_min = i
5         for j in range(i+1, n):
6             if L[j] < L[i_min]:
7                 i_min = j
8         L[i], L[i_min] = L[i_min], L[i]

```

6.5 Tri par insertion

Principe : on parcourt la liste et on insère chaque élément à sa place dans la partie déjà triée. Complexité : $O(n^2)$ en moyenne/pire cas, $O(n)$ si déjà trié. Stable, en place.

```

1 def tri_insertion(L):
2     for i in range(1, len(L)):
3         x = L[i]
4         j = i-1
5         while j >= 0 and L[j] > x:
6             L[j+1] = L[j]

```

```

7         j -= 1
8         L[j+1] = x

```

6.6 Tri à bulles

Principe : on parcourt la liste en échangeant les éléments adjacents mal ordonnés, jusqu'à ce que plus aucun échange ne soit nécessaire. Complexité : $O(n^2)$ (optimisé : $O(n)$ si déjà trié). Stable.

```

1 def tri_bulles(L):
2     n = len(L)
3     for i in range(n-1):
4         echange = False
5         for j in range(n-1-i):
6             if L[j+1] < L[j]:
7                 L[j], L[j+1] = L[j+1], L[j]
8                 echange = True
9         if not echange:
10            break

```

6.7 Tri rapide (Quick sort)

Principe récursif : choisir un pivot, partitionner la liste en éléments plus petits et plus grands, trier récursivement chaque partie. Complexité moyenne : $O(n \log n)$, pire cas : $O(n^2)$ (si pivot mal choisi). Non stable, en place possible.

```

1 def tri_rapide(L):
2     if len(L) <= 1:
3         return L
4     pivot = L[0]
5     petits = [x for x in L[1:] if x <= pivot]
6     grands = [x for x in L[1:] if x > pivot]
7     return tri_rapide(petits) + [pivot] + tri_rapide(grands)

```

6.8 Tri fusion (Merge sort)

Principe : diviser la liste en deux moitiés, les trier récursivement, puis fusionner les deux listes triées. Complexité : $O(n \log n)$ dans tous les cas. Stable, non en place.

```

1 def fusion(gauche, droite):
2     i = j = 0
3     res = []
4     while i < len(gauche) and j < len(droite):
5         if gauche[i] <= droite[j]:
6             res.append(gauche[i])
7             i += 1
8         else:
9             res.append(droite[j])
10            j += 1
11    res.extend(gauche[i:])
12    res.extend(droite[j:])
13    return res
14
15 def tri_fusion(L):
16     if len(L) <= 1:
17         return L
18     m = len(L)//2
19     return fusion(tri_fusion(L[:m]), tri_fusion(L[m:]))

```

6.9 Tri par tas (Heap sort)

Utilise une structure de tas binaire. Complexité : $O(n \log n)$ dans tous les cas, en place, non stable. (Voir chapitre sur les arbres binaires pour l'implémentation.)

6.10 Bilan des complexités

Algorithme	Meilleur cas	Moyenne	Pire cas	Stable
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	oui
Sélection	$O(n^2)$	$O(n^2)$	$O(n^2)$	non
Bulles	$O(n)$	$O(n^2)$	$O(n^2)$	oui
Rapide	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	non
Fusion	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	oui
Tas	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	non

6.11 Tri en Python : `sort()` et `sorted()`

La méthode `list.sort()` trie la liste en place, tandis que `sorted(L)` retourne une nouvelle liste triée. L'algorithme utilisé est un tri hybride (Timsort) mélangeant insertion et fusion, de complexité $O(n \log n)$.

7 Tuples, ensembles et dictionnaires

7.1 Les tuples (tuple)

Un tuple est une collection ordonnée et **immuable** d'éléments. Il se définit avec des parenthèses (optionnelles si non ambigu) et des virgules.

```

1 t1 = (1, 2, 3)
2 t2 = 4, 5, 6
3 t3 = ('a',)          # tuple à un élément (la virgule est nécessaire)
4 t4 = ()              # tuple vide

```

7.1.1 Propriétés

- Éléments ordonnés, accessibles par indice (comme les listes).
- Immuable : on ne peut pas modifier, ajouter ou supprimer d'élément.
- Peut contenir des types hétérogènes.
- Utilisé comme clé de dictionnaire (car immuable).

7.1.2 Opérations

```

1 t = (1, 2, 3)
2 len(t)          # 3
3 t[0]            # 1
4 t[-1]          # 3
5 t[1:3]          # (2, 3) (tranche)
6 (1,2) + (3,4)   # (1,2,3,4) concaténation
7 (1,2) * 3       # (1,2,1,2,1,2) répétition
8 3 in t          # True

```

7.1.3 Déballage (unpacking)

```

1 a, b, c = (10, 20, 30)  # a=10, b=20, c=30
2 x, *y = (1,2,3,4)       # x=1, y=[2,3,4] (le * capte le reste)

```

7.2 Les ensembles (set)

Un ensemble est une collection **non ordonnée**, **sans doublons**, et **muable**. On le définit avec des accolades ou `set()`.

```
1 s1 = {1, 2, 3, 2}          # {1, 2, 3} (les doublons sont éliminés)
2 s2 = set()                # ensemble vide (pas {} qui est un dict vide)
3 s3 = set([3,4,5])         # à partir d'une liste
```

7.2.1 Opérations de base

```
1 s = {1, 2, 3}
2 s.add(4)                  # ajoute un élément
3 s.remove(2)               # supprime 2 (erreur si absent)
4 s.discard(10)             # supprime sans erreur
5 s.pop()                  # retire et retourne un élément quelconque
6 s.clear()                 # vide l'ensemble
```

7.2.2 Opérations ensemblistes

- `s1.union(s2)` ou `s1 | s2` : union.
- `s1.intersection(s2)` ou `s1 & s2` : intersection.
- `s1.difference(s2)` ou `s1 - s2` : différence.
- `s1.symmetric_difference(s2)` ou `s1 ^ s2` : différence symétrique.
- `s1.issubset(s2)` : test de sous-ensemble.

Toutes ces méthodes retournent un nouvel ensemble.

```
1 a = {1,2,3}
2 b = {2,3,4}
3 a | b                      # {1,2,3,4}
4 a & b                      # {2,3}
5 a - b                      # {1}
6 b - a                      # {4}
7 a ^ b                      # {1,4}
```

7.3 Les dictionnaires (dict)

Un dictionnaire associe des **clés** (immuables) à des **valeurs** (quelconques). C'est une collection non ordonnée (mais l'ordre d'insertion est conservé en Python 3.7+).

7.3.1 Création

```
1 d1 = {}                   # dictionnaire vide
2 d2 = {"nom": "Dupont", "age": 20}
3 d3 = dict([("a",1), ("b",2)]) # à partir d'une liste de couples
4 d4 = {x: x**2 for x in range(5)} # par compréhension
```

7.3.2 Accès et modification

```
1 d = {"nom": "Dupont", "age": 20}
2 d["nom"]                      # "Dupont"
3 d["age"] = 21                 # modification
4 d["ville"] = "Paris"         # ajout d'une nouvelle clé
```

Si la clé n'existe pas, l'accès lève une erreur `KeyError`. Pour éviter cela, on utilise `get` :

```
1 d.get("nom")                  # "Dupont"
2 d.get("pays", "France")      # retourne "France" si "pays" absent
```


7.3.3 Suppression

```

1 d.pop("age")           # supprime et retourne la valeur
2 del d["nom"]           # supprime la clé
3 d.clear()              # vide le dictionnaire

```

7.3.4 Parcours

```

1 for cle in d:           # parcours des clés
2     print(cle, d[cle])
3
4 for valeur in d.values(): # parcours des valeurs
5     print(valeur)
6
7 for cle, valeur in d.items(): # parcours des couples
8     print(cle, valeur)

```

7.3.5 Test d'appartenance

```

1 if "age" in d:
2     print(d["age"])

```

7.3.6 Complexité des dictionnaires

Les dictionnaires Python sont implémentés par des tables de hachage. En moyenne :

- Accès, insertion, suppression : $O(1)$
- Dans le pire cas (beaucoup de collisions) : $O(n)$

8 Les fichiers

8.1 Ouverture et fermeture d'un fichier

La fonction `open(chemin, mode)` retourne un objet fichier. Le mode spécifie l'opération :

- 'r' : lecture (read), défaut.
- 'w' : écriture (write), écrase le fichier s'il existe, le crée sinon.
- 'a' : ajout (append), écrit à la fin du fichier.
- 'x' : création exclusive, échoue si le fichier existe déjà.

Pour les fichiers binaires, ajouter 'b' (ex. 'rb').

Il est impératif de fermer le fichier après utilisation (`f.close()`); mieux : utiliser `with` pour une fermeture automatique.

```

1 f = open("fichier.txt", "r")
2 contenu = f.read()
3 f.close()
4
5 # Version with (recommandée)
6 with open("fichier.txt", "r") as f:
7     contenu = f.read()

```

8.2 Lecture d'un fichier texte

Méthodes :

- `f.read()` : lit tout le fichier en une chaîne.
- `f.readline()` : lit une ligne (inclut le `\n` final).

- `f.readlines()` : lit toutes les lignes et retourne une liste.
- Parcours direct : `for ligne in f:` lit ligne par ligne (efficace).

```
1 with open("fichier.txt") as f:
2     for ligne in f:
3         print(ligne.rstrip())    # enlève le \n
```

8.3 Écriture dans un fichier texte

```
1 with open("sortie.txt", "w") as f:
2     f.write("Ligne 1\n")
3     f.write("Ligne 2\n")
```

`f.writelines(liste)` écrit une liste de chaînes (sans ajouter de `\n`).

8.4 Gestion des chemins

Sous Windows, les chemins contiennent des backslashes; il faut les échapper ou utiliser des raw strings `r"..."` ou des doubles backslashes, ou mieux, remplacer par des slashes.

```
1 chemin = "C:/Users/nom/Documents/fichier.txt"
```

8.5 Le module `os`

Fournit des fonctions pour interagir avec le système de fichiers.

```
1 import os
2 os.getcwd()           # répertoire courant
3 os.chdir(chemin)      # change de répertoire
4 os.listdir()          # liste des fichiers du répertoire
5 os.mkdir(nouveau_dossier) # crée un dossier
6 os.remove(fichier)    # supprime un fichier
7 os.rename(ancien, nouveau) # renomme
```

9 Bibliothèques mathématiques

9.1 Module `math`

Constantes et fonctions mathématiques usuelles.

```
1 import math
2 math.pi, math.e
3 math.sqrt(x)
4 math.exp(x), math.log(x), math.log10(x)
5 math.sin(x), math.cos(x), math.tan(x)
6 math.asin(x), math.acos(x), math.atan(x)
7 math.degrees(x), math.radians(x)
8 math.ceil(x), math.floor(x), math.fabs(x)
9 math.factorial(n)      # n!
10 math.comb(n, k)        # coefficient binomial
11 math.gcd(a, b)         # PGCD
```

9.2 Module `numpy`

Bibliothèque fondamentale pour le calcul scientifique. Elle introduit le type `ndarray` (tableau multidimensionnel) et des opérations vectorisées.

9.2.1 Création de tableaux

```

1 import numpy as np
2 a = np.array([1,2,3])           # tableau 1D
3 b = np.array([[1,2],[3,4]])     # tableau 2D (matrice)
4 c = np.zeros((3,4))            # matrice de zéros 3x4
5 d = np.ones((2,3))             # matrice de uns
6 e = np.eye(5)                  # matrice identité 5x5
7 f = np.arange(0,10,2)          # [0,2,4,6,8]
8 g = np.linspace(0,1,5)         # [0, 0.25, 0.5, 0.75, 1]

```

9.2.2 Propriétés et accès

```

1 a.shape           # dimensions du tableau
2 a.ndim            # nombre de dimensions
3 a.size            # nombre total d'éléments
4 a[0]              # premier élément
5 b[1,1]            # élément ligne 1 colonne 1
6 b[:,0]            # première colonne
7 b[0,:]            # première ligne

```

Les tranches (slicing) fonctionnent comme pour les listes, mais renvoient des **vues** (pas de copie).

9.2.3 Opérations vectorisées

```

1 a + 5             # addition scalaire (à chaque élément)
2 2 * a             # multiplication scalaire
3 a + b             # addition terme à terme (si dimensions compatibles)
4 a * b             # multiplication terme à terme
5 np.dot(a,b)       # produit matriciel (ou produit scalaire si vecteurs)
6 a @ b             # produit matriciel (Python 3.5+)
7 np.transpose(b)   # transposée
8 b.T               # transposée (propriété)

```

Les fonctions mathématiques de numpy s'appliquent élément par élément :

```

1 np.sqrt(a)        # racine carrée de chaque élément
2 np.exp(a)         # exponentielle
3 np.log(a)         # logarithme
4 np.sin(a)         # sinus

```

9.2.4 Algèbre linéaire avec numpy.linalg

```

1 from numpy.linalg import inv, det, eig, solve, matrix_power
2 inv(A)             # inverse de A
3 det(A)             # déterminant
4 eig(A)             # valeurs propres et vecteurs propres
5 solve(A, b)        # résolution de A x = b
6 matrix_power(A, n) # puissance n de A

```

9.3 Module random

Génération de nombres aléatoires.

```

1 import random
2 random.randint(1,10) # entier entre 1 et 10 inclus
3 random.uniform(0,1) # flottant entre 0 et 1
4 random.random()     # flottant dans [0,1[

```

```

5 random.choice(['a','b']) # choix aléatoire dans une liste
6 random.shuffle(liste)   # mélange aléatoire en place

```

Avec `numpy.random` :

```

1 import numpy.random as npr
2 npr.randint(0,10,5)      # 5 entiers entre 0 et 9
3 npr.random((3,2))        # tableau 3x2 de flottants dans [0,1[
4 npr.normal(0,1,10)       # 10 tirages selon loi normale N(0,1)

```

9.4 Module matplotlib

Tracés de courbes et visualisation.

```

1 import matplotlib.pyplot as plt
2 x = np.linspace(0, 2*np.pi, 100)
3 y = np.sin(x)
4 plt.plot(x, y, label='sin(x)', color='red', linestyle='--')
5 plt.xlabel('x')
6 plt.ylabel('y')
7 plt.title('Fonction sinus')
8 plt.legend()
9 plt.grid()
10 plt.show()

```

Autres fonctions : `plt.scatter` (nuage de points), `plt.bar` (diagramme en barres), `plt.contour` (courbes de niveau), `plt.imshow` (affichage d'image). On peut sauvegarder avec `plt.savefig('nom.png')`.

9.5 Module scipy (complément)

Bibliothèque de calcul scientifique avancé.

```

1 import scipy.integrate as integrate
2 integrate.quad(f, a, b)                # intégration numérique de f sur [a,b]
3 from scipy.optimize import fsolve, root
4 fsolve(f, x0)                          # recherche de zéro (scalaire)
5 root(f, x0)                            # recherche de zéro (système)
6 from scipy.integrate import odeint
7 odeint(f, y0, t)                       # résolution d'équation différentielle

```

10 Les arbres binaires

10.1 Définitions

Un **arbre binaire** est une structure hiérarchique où chaque nœud possède au plus deux fils : le sous-arbre gauche et le sous-arbre droit.

- **Racine** : nœud initial, sans père.
- **Feuille** : nœud sans fils.
- **Profondeur** d'un nœud : nombre d'arêtes de la racine à ce nœud.
- **Hauteur** de l'arbre : profondeur maximale d'une feuille.
- **Degré** d'un nœud : nombre de ses fils (0,1,2).

10.2 Représentation en Python

On peut représenter un arbre binaire par une liste [valeur, gauche, droit], où gauche et droit sont eux-mêmes des listes (ou None pour vide).

```

1 # Exemple d'arbre :
2 #           5
3 #         /  \
4 #        3    7
5 #       / \   \
6 #      1  4   9
7
8 arbre = [5,
9          [3, [1, None, None], [4, None, None]],
10         [7, None, [9, None, None]]]
```

10.3 Parcours d'un arbre binaire

10.3.1 Parcours préfixe (racine, gauche, droit)

```

1 def prefixe(a):
2     if a is None:
3         return
4     print(a[0], end=' ')
5     prefixe(a[1])
6     prefixe(a[2])
7 # -> 5 3 1 4 7 9
```

10.3.2 Parcours infixe (gauche, racine, droit)

```

1 def infixe(a):
2     if a is None:
3         return
4     infixe(a[1])
5     print(a[0], end=' ')
6     infixe(a[2])
7 # -> 1 3 4 5 7 9
```

10.3.3 Parcours postfixe (gauche, droit, racine)

```

1 def postfixe(a):
2     if a is None:
3         return
4     postfixe(a[1])
5     postfixe(a[2])
6     print(a[0], end=' ')
7 # -> 1 4 3 9 7 5
```

10.3.4 Parcours en largeur (BFS)

Utilise une file.

```

1 from collections import deque
2 def largeur(racine):
3     if racine is None:
4         return
5     file = deque([racine])
6     while file:
```

```

7     noeud = file.popleft()
8     print(noeud[0], end=' ')
9     if noeud[1] is not None:
10        file.append(noeud[1])
11    if noeud[2] is not None:
12        file.append(noeud[2])
13 # -> 5 3 7 1 4 9

```

10.4 Arbres binaires de recherche (ABR)

Un ABR est un arbre binaire tel que pour chaque nœud de valeur v :

- toutes les valeurs du sous-arbre gauche sont strictement inférieures à v ;
- toutes les valeurs du sous-arbre droit sont supérieures ou égales à v (selon la convention).

10.4.1 Recherche dans un ABR

```

1 def rechercher(arbre, x):
2     if arbre is None:
3         return False
4     if x == arbre[0]:
5         return True
6     elif x < arbre[0]:
7         return rechercher(arbre[1], x)
8     else:
9         return rechercher(arbre[2], x)

```

Complexité : $O(h)$ où h est la hauteur de l'arbre.

10.4.2 Insertion dans un ABR

```

1 def inserer(arbre, x):
2     if arbre is None:
3         return [x, None, None]
4     if x < arbre[0]:
5         arbre[1] = inserer(arbre[1], x)
6     else:
7         arbre[2] = inserer(arbre[2], x)
8     return arbre

```

10.5 Tas (heap)

Un **tas** est un arbre binaire **complet** (tous les niveaux sont remplis, sauf éventuellement le dernier, rempli de gauche à droite) qui vérifie une propriété d'ordre :

- **Tas-max** : la valeur d'un nœud est supérieure ou égale à celles de ses fils.
- **Tas-min** : la valeur d'un nœud est inférieure ou égale à celles de ses fils.

10.5.1 Représentation par tableau

Grâce à la complétude, on peut représenter un tas par un tableau (liste) :

- La racine est à l'indice 0.
- Pour un nœud à l'indice i , son fils gauche est à $2i + 1$, son fils droit à $2i + 2$, son parent à $\lfloor (i - 1)/2 \rfloor$.

10.5.2 Tamisage (heapify)

Rétablit la propriété de tas à partir d'un nœud donné.

```

1 def tamiser(tas, n, i):
2     # n = taille du tas, i = indice à tamiser (tas-max)
3     while True:
4         plus_grand = i
5         g = 2*i+1
6         d = 2*i+2
7         if g < n and tas[g] > tas[plus_grand]:
8             plus_grand = g
9         if d < n and tas[d] > tas[plus_grand]:
10            plus_grand = d
11        if plus_grand == i:
12            break
13        tas[i], tas[plus_grand] = tas[plus_grand], tas[i]
14        i = plus_grand

```

10.5.3 Construction d'un tas

```

1 def construire_tas(tas):
2     n = len(tas)
3     for i in range(n//2-1, -1, -1):
4         tamiser(tas, n, i)

```

10.5.4 Tri par tas (Heap sort)

```

1 def tri_tas(t):
2     n = len(t)
3     construire_tas(t)
4     for i in range(n-1, 0, -1):
5         t[0], t[i] = t[i], t[0]
6         tamiser(t, i, 0)

```

Complexité $O(n \log n)$, en place, non stable.

10.5.5 File de priorité avec tas

Un tas permet d'implémenter une file de priorité : l'élément de priorité maximale (ou minimale) est toujours à la racine. Insertion en $O(\log n)$ par ajout en fin et remontée (percolation vers le haut).

11 Les graphes

11.1 Définitions générales

Un **graphe** est une structure de données constituée d'un ensemble de **sommets** (ou nœuds) et d'un ensemble de **liens** entre ces sommets. On distingue :

- **Graphe non orienté** : les liens sont des **arêtes** (paires non ordonnées $\{u, v\}$).
- **Graphe orienté** : les liens sont des **arcs** (couples ordonnés (u, v)).

Un graphe peut être **pondéré** si chaque lien possède un poids (nombre réel). L'**ordre** d'un graphe est son nombre de sommets. La **taille** est son nombre d'arêtes/arcs.

11.2 Vocabulaire

- **Chemin** (orienté) : suite de sommets $(v_0, v_1, \dots, v_k)$ telle que chaque (v_i, v_{i+1}) est un arc.
- **Chaîne** (non orienté) : suite de sommets telle que chaque $\{v_i, v_{i+1}\}$ est une arête.
- **Cycle** : chaîne (ou chemin) dont le premier et le dernier sommet sont identiques.
- **Degré** (non orienté) : nombre d'arêtes incidentes à un sommet.

- **Degré sortant / entrant** (orienté) : nombre d'arcs partant / arrivant à un sommet.
- **Graphe connexe** (non orienté) : pour toute paire de sommets, il existe une chaîne les reliant.

11.3 Représentations en Python

11.3.1 Matrice d'adjacence

Matrice carrée M de taille $n \times n$ où $M[i][j] = 1$ s'il existe un arc de i vers j , 0 sinon. Pour un graphe pondéré, on stocke le poids (et une valeur spéciale comme l'infini pour absence d'arc). Avantage : accès direct à l'existence d'un lien. Inconvénient : occupation mémoire $O(n^2)$, parcours des voisins en $O(n)$.

11.3.2 Liste d'adjacence

Pour chaque sommet, on stocke la liste de ses voisins (ou des couples (voisin, poids)). Mémoire $O(n+m)$. Accès rapide aux voisins.

```
1 # Liste d'adjacence (graphe non orienté)
2 L = [[1], [0,2,3], [1], [1]]
3 # Avec dictionnaire pour étiquettes
4 G = {
5     'A': ['B', 'C'],
6     'B': ['A', 'D'],
7     'C': ['A'],
8     'D': ['B']
9 }
```

11.4 Parcours de graphes

11.4.1 Parcours en largeur (BFS)

Explore les sommets par niveaux de distance croissante depuis un sommet source. Utilise une file. Permet de calculer les distances (nombre d'arêtes) dans un graphe non pondéré. Complexité $O(n+m)$.

```
1 from collections import deque
2
3 def bfs(G, depart):
4     n = len(G)
5     visites = [False]*n
6     distances = [-1]*n
7     file = deque([depart])
8     visites[depart] = True
9     distances[depart] = 0
10    while file:
11        u = file.popleft()
12        for v in G[u]:
13            if not visites[v]:
14                visites[v] = True
15                distances[v] = distances[u] + 1
16                file.append(v)
17    return distances
```

11.4.2 Parcours en profondeur (DFS)

Explore en allant le plus loin possible avant de revenir en arrière. Utilise une pile (ou récursivité). Complexité $O(n+m)$.

```
1 def dfs_rec(G, u, visites):
2     visites[u] = True
3     print(u, end=' ')
4     for v in G[u]:
```



```

5         if not visites[v]:
6             dfs_rec(G, v, visites)
7
8 def dfs(G, depart):
9     visites = [False]*len(G)
10    dfs_rec(G, depart, visites)

```

11.5 Détection de cycles

11.5.1 Graphe non orienté

Lors d'un DFS, si on rencontre un voisin déjà visité qui n'est pas le parent, il y a un cycle.

11.5.2 Graphe orienté

On utilise trois états : 0 = non visité, 1 = en cours de traitement, 2 = traité. Si on rencontre un sommet en cours, cycle.

11.6 Plus courts chemins – Dijkstra

Pour graphe pondéré à poids positifs. Calcule les distances minimales depuis une source.

```

1 import heapq
2
3 def dijkstra(G, source):
4     n = len(G)
5     dist = [float('inf')]*n
6     dist[source] = 0
7     prev = [None]*n
8     pq = [(0, source)]
9     while pq:
10        d, u = heapq.heappop(pq)
11        if d > dist[u]:
12            continue
13        for v, w in G[u]:
14            nd = d + w
15            if nd < dist[v]:
16                dist[v] = nd
17                prev[v] = u
18                heapq.heappush(pq, (nd, v))
19    return dist, prev

```

11.7 Plus courts chemins tous couples – Floyd-Warshall

Pour graphe orienté, poids quelconques (pas de cycle négatif). Complexité $O(n^3)$.

```

1 def floyd_warshall(M):
2     n = len(M)
3     dist = [[M[i][j] for j in range(n)] for i in range(n)]
4     for k in range(n):
5         for i in range(n):
6             for j in range(n):
7                 if dist[i][k] + dist[k][j] < dist[i][j]:
8                     dist[i][j] = dist[i][k] + dist[k][j]
9    return dist

```

12 Programmation dynamique

12.1 Principes fondamentaux

La programmation dynamique est une méthode de résolution de problèmes d'optimisation qui combine :

- **Sous-structure optimale** : une solution optimale du problème contient des solutions optimales pour les sous-problèmes.
- **Chevauchement des sous-problèmes** : les mêmes sous-problèmes sont rencontrés plusieurs fois ; on évite de les recalculer en stockant leurs résultats.

Deux approches :

- **Top-down (mémoïsation)** : récursif avec stockage des résultats déjà calculés (dictionnaire).
- **Bottom-up (tabulation)** : itératif, on remplit un tableau à partir des cas de base.

12.2 Exemple : suite de Fibonacci

```

1 # Naïf (exponentiel)
2 def fibo_naif(n):
3     if n <= 1:
4         return 1
5     return fibo_naif(n-1) + fibo_naif(n-2)
6
7 # Top-down (mémoïsation)
8 memo = {}
9 def fibo_td(n):
10    if n in memo:
11        return memo[n]
12    if n <= 1:
13        return 1
14    memo[n] = fibo_td(n-1) + fibo_td(n-2)
15    return memo[n]
16
17 # Bottom-up
18 def fibo_bu(n):
19    if n <= 1:
20        return 1
21    t = [0]*(n+1)
22    t[0] = t[1] = 1
23    for i in range(2, n+1):
24        t[i] = t[i-1] + t[i-2]
25    return t[n]
```

12.3 Problème du rendu de monnaie

Étant donné un ensemble de pièces (valeurs) et une somme S , trouver le nombre minimal de pièces.
Relation : $f(0) = 0$, $f(s) = \min_{c \leq s} (1 + f(s - c))$.

```

1 def rendu_monnaie(pieces, somme):
2     INF = float('inf')
3     nb = [0] + [INF]*somme
4     for s in range(1, somme+1):
5         for p in pieces:
6             if p <= s:
7                 nb[s] = min(nb[s], 1 + nb[s-p])
8     return nb[somme]
```

12.4 Problème du sac à dos 0/1

Objets avec poids w_i et valeur v_i , capacité C . Maximiser la valeur. Relation : $f(i, c) = \max(f(i-1, c), v_i + f(i-1, c-w_i))$ si $w_i \leq c$.

```

1 def sac_dos(poids, valeurs, C):
2     n = len(poids)
3     dp = [[0]*(C+1) for _ in range(n+1)]
4     for i in range(1, n+1):
5         for c in range(1, C+1):
6             if poids[i-1] <= c:
7                 dp[i][c] = max(dp[i-1][c],
8                               dp[i-1][c-poids[i-1]] + valeurs[i-1])
9             else:
10                dp[i][c] = dp[i-1][c]
11    return dp[n][C]
```

12.5 Plus longue sous-séquence commune (LCS)

Étant donné deux chaînes X et Y , trouver la plus longue sous-séquence commune. Relation : $L[i][j] =$

$$\begin{cases} 0 & i = 0 \text{ ou } j = 0 \\ L[i-1][j-1] + 1 & X[i-1] = Y[j-1] \\ \max(L[i-1][j], L[i][j-1]) & \text{sinon} \end{cases}$$

```

1 def lcs(X, Y):
2     m, n = len(X), len(Y)
3     L = [[0]*(n+1) for _ in range(m+1)]
4     for i in range(1, m+1):
5         for j in range(1, n+1):
6             if X[i-1] == Y[j-1]:
7                 L[i][j] = L[i-1][j-1] + 1
8             else:
9                 L[i][j] = max(L[i-1][j], L[i][j-1])
10    # reconstruction
11    i, j = m, n
12    seq = []
13    while i>0 and j>0:
14        if X[i-1] == Y[j-1]:
15            seq.append(X[i-1])
16            i -= 1; j -= 1
17        elif L[i-1][j] > L[i][j-1]:
18            i -= 1
19        else:
20            j -= 1
21    return ''.join(reversed(seq))
```

Complexité $O(mn)$.

12.6 Distance de Levenshtein

La **distance de Levenshtein** (ou distance d'édition) entre deux chaînes X et Y est le nombre minimal d'opérations élémentaires (insertion, suppression, substitution) nécessaires pour transformer X en Y .

12.6.1 Relation de récurrence

Soit $D(i, j)$ la distance entre les i premiers caractères de X et les j premiers caractères de Y .

$$D(i, j) = \begin{cases} i & \text{si } j = 0 \\ j & \text{si } i = 0 \\ D(i-1, j-1) & \text{si } X[i-1] = Y[j-1] \\ 1 + \min(D(i-1, j), D(i, j-1), D(i-1, j-1)) & \text{sinon.} \end{cases}$$

12.6.2 Tabulation (Bottom-Up)

```

1 def levenshtein_tab(X, Y):
2     n, m = len(X), len(Y)
3     M = [[0] * (m + 1) for _ in range(n + 1)]
4     for i in range(n + 1):
5         for j in range(m + 1):
6             if j == 0:
7                 M[i][j] = i
8             elif i == 0:
9                 M[i][j] = j
10            elif X[i-1] == Y[j-1]:
11                M[i][j] = M[i-1][j-1]
12            else:
13                M[i][j] = 1 + min(M[i-1][j], M[i][j-1], M[i-1][j-1])
14    return M[n][m]
```

Complexité $O(mn)$.

12.7 Autres exemples classiques

- Partition équilibrée d'un ensemble.
- Plus longue sous-chaîne commune.
- Algorithme de Floyd-Warshall (déjà traité avec les graphes).

13 Traitement des images**13.1** Introduction aux images numériques

Une image numérique est une matrice de **pixels** (picture elements). Chaque pixel est une valeur (ou un triplet de valeurs) représentant une couleur. On distingue trois types principaux :

- **Image binaire** : chaque pixel vaut 0 (noir) ou 1 (blanc). Elle occupe 1 bit par pixel.
- **Image en niveaux de gris** : chaque pixel est un entier compris entre 0 (noir) et 255 (blanc). Elle occupe 1 octet par pixel.
- **Image couleur** : chaque pixel est un triplet (R, V, B) où chaque composante est entre 0 et 255. Elle occupe 3 octets par pixel.

La **résolution** d'une image est le nombre de pixels en largeur $\times$ hauteur (ex. 1920×1080). La **taille mémoire** en octets est résolution $\times$ nombre d'octets par pixel.

13.2 Chargement et affichage d'une image avec Matplotlib

Pour manipuler des images en Python, on utilise souvent les bibliothèques `matplotlib` et `numpy`. L'image est chargée sous forme d'un tableau `numpy` (hauteur, largeur, 3) pour une couleur, ou (hauteur, largeur) pour une image en niveaux de gris.

```

1 import matplotlib.pyplot as plt
2 import matplotlib.image as mpi
3 import numpy as np
4
5 # Charger l'image depuis un fichier
6 img = mpi.imread("chemin/vers/image.jpg")
7
8 # Afficher l'image
9 plt.imshow(img)
10 plt.axis('off') # pour ne pas afficher les axes
11 plt.show()

```

Pour les images en niveaux de gris, on peut préciser `cmap='gray'` :

```

1 plt.imshow(img_gris, cmap='gray')

```

13.3 Opérations de base sur les images

Toutes ces opérations consistent à créer une nouvelle image (même taille) et à remplir chaque pixel en fonction des pixels de l'image originale.

13.3.1 Négatif

Pour une image en niveaux de gris, le négatif est obtenu par $255 - \text{pixel}$. Pour une image couleur, on applique la même formule à chaque composante.

```

1 def negatif(img):
2     """Retourne le négatif d'une image (couleur ou niveaux de gris)."""
3     return 255 - img

```

Si l'image est de type entier non signé (uint8), attention aux débordements ; la soustraction fonctionne car numpy gère les dépassements par modulo, mais il vaut mieux convertir en entier signé temporairement ou utiliser $255 - \text{img}$ directement car les valeurs sont dans $[0, 255]$.

13.3.2 Inversion verticale

L'inversion verticale (miroir haut-bas) s'obtient en parcourant les lignes de bas en haut.

```

1 def inverser_verticale(img):
2     """Inverse l'image verticalement (effet miroir haut-bas)."""
3     return img[::-1, :, :] # pour une image couleur
4     # Pour niveaux de gris : img[::-1, :]

```

13.3.3 Miroir horizontal

L'effet miroir gauche-droite s'obtient en inversant l'ordre des colonnes.

```

1 def miroir_horizontal(img):
2     """Effet miroir gauche-droite."""
3     return img[:, ::-1, :]

```

13.3.4 Rotation à 90°

Une rotation de 90° dans le sens horaire combine une transposition et un miroir vertical.

```

1 def rotation_90(img):
2     """Rotation de 90 deg dans le sens horaire."""
3     return img.transpose(1,0,2)[::-1, :, :] # transposition puis miroir
        vertical

```

Explication : `transpose(1,0,2)` échange lignes et colonnes, puis `[::-1, ...]` inverse les lignes pour obtenir la rotation horaire.

13.3.5 Extraction d'une moitié

Pour extraire la moitié droite de l'image, on sélectionne les colonnes à partir de la moitié.

```
1 def moitie_droite(img):
2     """Retourne la moitié droite de l'image."""
3     p = img.shape[1]
4     return img[:, p//2:, :]
```

De même pour la moitié gauche, haute, basse.

13.4 Manipulation des couleurs**13.4.1 Conversion en niveaux de gris**

La conversion d'une image couleur en niveaux de gris utilise la formule de luminance :

$$Y = 0.299R + 0.587G + 0.114B$$

On peut implémenter cette formule élément par élément.

```
1 def gris(img):
2     """Convertit une image couleur en niveaux de gris."""
3     return 0.299*img[:, :, 0] + 0.587*img[:, :, 1] + 0.114*img[:, :, 2]
```

Le résultat est un tableau 2D de flottants entre 0 et 255. On peut le convertir en entier non signé avec `astype(np.uint8)`.

13.4.2 Conversion en noir et blanc (seuillage)

À partir de l'image en niveaux de gris, on fixe un seuil. Si la valeur est supérieure au seuil, le pixel devient blanc (255), sinon noir (0). On peut aussi produire une image binaire (0/1).

```
1 def noir_et_blanc(img, seuil=128):
2     """Seuillage : image binaire (0 ou 255)."""
3     gris_img = gris(img) # si l'image est couleur
4     return (gris_img > seuil).astype(np.uint8) * 255
```

L'image résultat est en niveaux de gris mais ne contient que deux valeurs.

13.4.3 Extraction des canaux RVB

On peut isoler chaque canal pour visualiser la contribution du rouge, vert ou bleu.

```
1 def extraire_canaux(img):
2     """Retourne les trois images correspondant aux canaux R, V, B."""
3     n, p, _ = img.shape
4     rouge = np.zeros((n,p,3), dtype=np.uint8)
5     vert  = np.zeros((n,p,3), dtype=np.uint8)
6     bleu  = np.zeros((n,p,3), dtype=np.uint8)
7     rouge[:, :, 0] = img[:, :, 0]
8     vert[:, :, 1]  = img[:, :, 1]
9     bleu[:, :, 2]  = img[:, :, 2]
10    return rouge, vert, bleu
```

On peut aussi afficher chaque canal en niveaux de gris en prenant simplement la composante correspondante.

13.5 Redimensionnement d'images**13.5.1 Agrandissement par répétition**

Pour agrandir une image d'un facteur entier t , on répète chaque pixel t fois dans chaque direction. Cela donne un effet de pixelisation.

```

1 def agrandir(img, t):
2     """Agrandit l'image par un facteur t (répétition de pixels)."""
3     n, p = img.shape[0], img.shape[1]
4     if img.ndim == 3:
5         res = np.zeros((n*t, p*t, 3), dtype=img.dtype)
6         for i in range(n*t):
7             for j in range(p*t):
8                 res[i,j] = img[i//t, j//t]
9     else: # niveaux de gris
10        res = np.zeros((n*t, p*t), dtype=img.dtype)
11        for i in range(n*t):
12            for j in range(p*t):
13                res[i,j] = img[i//t, j//t]
14    return res

```

On peut aussi utiliser les opérations vectorisées de numpy pour plus d'efficacité :

```

1 def agrandir_fast(img, t):
2     n, p = img.shape[0], img.shape[1]
3     return np.repeat(np.repeat(img, t, axis=0), t, axis=1)

```

13.5.2 Réduction par moyennage

Pour réduire une image d'un facteur t , on divise l'image en blocs $t \times t$ et on calcule la moyenne des pixels de chaque bloc. C'est une méthode de sous-échantillonnage avec anti-crénelage.

```

1 def reduire(img, t):
2     """Réduit l'image par un facteur t en moyennant les blocs t x t."""
3     n, p = img.shape[0], img.shape[1]
4     n2, p2 = n//t, p//t
5     if img.ndim == 3:
6         res = np.zeros((n2, p2, 3), dtype=np.uint8)
7         for i in range(n2):
8             for j in range(p2):
9                 bloc = img[i*t:(i+1)*t, j*t:(j+1)*t, :]
10                res[i,j] = bloc.mean(axis=(0,1))
11    else:
12        res = np.zeros((n2, p2), dtype=np.uint8)
13        for i in range(n2):
14            for j in range(p2):
15                bloc = img[i*t:(i+1)*t, j*t:(j+1)*t]
16                res[i,j] = bloc.mean()
17    return res

```

La fonction `mean` retourne des flottants ; on les convertit en entier avec `astype(np.uint8)` si nécessaire.

13.6 Superposition d'images (mélange)

On peut superposer deux images de même taille en effectuant une combinaison linéaire : $I = \alpha I_1 + (1 - \alpha) I_2$, avec α entre 0 et 1.

```

1 def superposer(img1, img2, alpha=0.5):
2     """Superpose deux images avec un coefficient alpha."""
3     return (alpha * img1 + (1-alpha) * img2).astype(np.uint8)

```

Les deux images doivent avoir les mêmes dimensions et le même nombre de canaux.

13.7 Filtrage par convolution

Le filtrage par convolution consiste à appliquer un noyau (matrice) à chaque pixel en effectuant la somme pondérée des pixels voisins. C'est une opération fondamentale pour le lissage, la détection de contours, etc.

Voici une implémentation simple pour une image en niveaux de gris :

```

1 def convolution(img, noyau):
2     """Applique un noyau de convolution à une image en niveaux de gris."""
3     h, w = img.shape
4     kh, kw = noyau.shape
5     ph, pw = kh//2, kw//2
6     res = np.zeros_like(img)
7     # On traite seulement les pixels où le noyau est entièrement dans
8     # l'image
9     for i in range(ph, h-ph):
10        for j in range(pw, w-pw):
11            zone = img[i-ph:i+ph+1, j-pw:j+pw+1]
12            res[i,j] = np.sum(zone * noyau)
13    return res

```

Pour une image couleur, on applique la convolution à chaque canal séparément.

Exemples de noyaux :

— Lissage (moyenleur) : $\frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$

— Flou gaussien (approximation)

— Détection de contours Sobel : $G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$

14 Méthodes numériques

14.1 Calculs approchés d'intégrales

On subdivise $[a, b]$ en n sous-intervalles de pas $h = \frac{b-a}{n}$, $x_i = a + ih$.

14.1.1 Méthode des rectangles

- Rectangles à gauche : $I \approx h \sum_{i=0}^{n-1} f(x_i)$.
- Rectangles à droite : $I \approx h \sum_{i=1}^n f(x_i)$.
- Point milieu : $I \approx h \sum_{i=0}^{n-1} f\left(x_i + \frac{h}{2}\right)$.

```

1 def rectangles_gauche(f, a, b, n):
2     h = (b-a)/n
3     return h * sum(f(a + i*h) for i in range(n))

```

14.1.2 Méthode des trapèzes

$$I \approx h \left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(a + ih) \right)$$

```

1 def trapezes(f, a, b, n):
2     h = (b-a)/n
3     s = 0.5*(f(a)+f(b))
4     for i in range(1, n):

```



```

5         s += f(a + i*h)
6     return h * s

```

14.1.3 Méthode de Simpson

Nécessite n pair. Approximation par paraboles :

$$I \approx \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{i=1}^{n/2} f(a + (2i-1)h) + 2 \sum_{i=1}^{n/2-1} f(a + 2ih) \right]$$

```

1 def simpson(f, a, b, n):
2     if n % 2 != 0:
3         raise ValueError("n doit être pair")
4     h = (b-a)/n
5     s = f(a) + f(b)
6     for i in range(1, n):
7         coeff = 4 if i%2 == 1 else 2
8         s += coeff * f(a + i*h)
9     return h/3 * s

```

14.2 Résolution approchée de $f(x) = 0$

14.2.1 Méthode de dichotomie

Pour f continue sur $[a, b]$ avec $f(a)f(b) \leq 0$. On coupe l'intervalle en deux jusqu'à obtenir une précision ε . Complexité : $O(\log \frac{b-a}{\varepsilon})$.

```

1 def dichotomie(f, a, b, eps):
2     while b - a > eps:
3         m = (a+b)/2
4         if f(m) == 0:
5             return m
6         elif f(a)*f(m) < 0:
7             b = m
8         else:
9             a = m
10    return (a+b)/2

```

14.2.2 Méthode de Newton-Raphson

On itère $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$. Convergence quadratique si proche de la racine.

```

1 def derivee(f, x, h=1e-5):
2     return (f(x+h) - f(x-h)) / (2*h)
3
4 def newton(f, x0, eps, max_iter=100):
5     x = x0
6     for _ in range(max_iter):
7         fx = f(x)
8         if abs(fx) < eps:
9             return x
10        df = derivee(f, x)
11        if df == 0:
12            raise ValueError("Dérivée nulle")
13        x -= fx / df
14    return x

```

14.3 Équations différentielles ordinaires

Soit $y'(t) = f(t, y(t))$ avec $y(t_0) = y_0$. On cherche $y(t)$ pour $t \in [t_0, t_0 + T]$.

14.3.1 Méthode d'Euler explicite

Discretisation avec pas $h = T/n$, $t_k = t_0 + kh$. Schéma :

$$y_{k+1} = y_k + hf(t_k, y_k)$$

```

1 def euler_explicite(f, y0, t0, T, n):
2     h = T/n
3     t = t0
4     y = y0
5     res_t = [t0]
6     res_y = [y0]
7     for _ in range(n):
8         y += h * f(t, y)
9         t += h
10        res_t.append(t)
11        res_y.append(y)
12    return res_t, res_y

```

14.3.2 Méthode de Runge-Kutta d'ordre 4 (RK4)

Plus précise qu'Euler.

```

1 def rk4(f, y0, t0, T, n):
2     h = T/n
3     t = t0
4     y = y0
5     res_t = [t0]
6     res_y = [y0]
7     for _ in range(n):
8         k1 = h * f(t, y)
9         k2 = h * f(t + h/2, y + k1/2)
10        k3 = h * f(t + h/2, y + k2/2)
11        k4 = h * f(t + h, y + k3)
12        y += (k1 + 2*k2 + 2*k3 + k4) / 6
13        t += h
14        res_t.append(t)
15        res_y.append(y)
16    return res_t, res_y

```

14.3.3 Utilisation de `scipy.integrate.odeint`

```

1 from scipy.integrate import odeint
2 t = np.linspace(t0, t0+T, n+1)
3 y = odeint(f, y0, t)

```

15 Complexité algorithmique

15.1 Notations asymptotiques

Pour comparer l'efficacité des algorithmes indépendamment de la machine, on utilise les notations asymptotiques qui décrivent le comportement du temps d'exécution (ou de l'espace mémoire) lorsque la taille n des données devient grande.

- **Notation grand O (O)** : majorant asymptotique. On dit que $f(n) = O(g(n))$ s'il existe $c > 0$ et n_0 tels que $|f(n)| \leq c|g(n)|$ pour tout $n \geq n_0$.
- **Notation grand Omega (Ω)** : minorant asymptotique. $f(n) = \Omega(g(n))$ s'il existe $c > 0$ et n_0 tels que $|f(n)| \geq c|g(n)|$ pour tout $n \geq n_0$.
- **Notation grand Theta (Θ)** : encadrement asymptotique. $f(n) = \Theta(g(n))$ si $f(n) = O(g(n))$ et $f(n) = \Omega(g(n))$.

15.2 Règles de calcul de complexité

- Chaque instruction élémentaire (affectation, opération arithmétique, comparaison) a un coût $O(1)$.
- La complexité d'une séquence d'instructions est la somme des complexités.
- Pour une boucle **for** exécutée n fois, la complexité est $n \times$ (complexité du corps).
- Pour une boucle **while**, on doit déterminer le nombre d'itérations à l'aide d'un variant.
- Pour une fonction récursive, on établit une relation de récurrence.

15.3 Exemples de calcul

```

1 def boucle_for(n):
2     s = 0
3     for i in range(n):
4         s += i
5     return s
6 # Complexité :  $O(1) + O(n) \cdot O(1) = O(n)$ 

```

```

1 def boucles_imbriquees(n):
2     s = 0
3     for i in range(n):
4         for j in range(n):
5             s += i*j
6     return s
7 # Complexité :  $O(n) * O(n) * O(1) = O(n^2)$ 

```

```

1 def recherche_dicho(L, x):
2     g, d = 0, len(L)-1
3     while g <= d:
4         m = (g+d)//2
5         if L[m] == x:
6             return m
7         elif L[m] < x:
8             g = m+1
9         else:
10            d = m-1
11    return -1
12 # À chaque itération, la taille de la zone de recherche est divisée par 2.
13 # Complexité :  $O(\log n)$ 

```

15.4 Ordres de grandeur usuels

Complexité	Exemple
$O(1)$	accès à un élément de tableau
$O(\log n)$	recherche dichotomique
$O(n)$	parcours simple
$O(n \log n)$	tri fusion, tri rapide (moyenne)
$O(n^2)$	tri à bulles, tri par sélection
$O(2^n)$	Fibonacci récursif naïf

15.5 Théorème maître

Pour les algorithmes récursifs de type "diviser pour régner" de la forme $T(n) = aT(n/b) + O(n^d)$ (avec $a \geq 1$, $b > 1$, $d \geq 0$), on a :

$$T(n) = \begin{cases} O(n^d) & \text{si } d > \log_b a \\ O(n^d \log n) & \text{si } d = \log_b a \\ O(n^{\log_b a}) & \text{si } d < \log_b a \end{cases}$$

Exemple : tri fusion $T(n) = 2T(n/2) + O(n) \rightarrow d = 1$, $\log_2 2 = 1 \rightarrow T(n) = O(n \log n)$.

16 Algorithmes gloutons

16.1 Principe

Un algorithme glouton construit une solution en faisant à chaque étape le meilleur choix local, en espérant que cela mène à une solution globale optimale.

16.2 Caractéristiques

- **Choix glouton** : on sélectionne l'élément le plus prometteur selon un critère.
- **Admissibilité** : on vérifie que le choix ne viole pas les contraintes.
- **Complexité** : souvent faible (tri + parcours).

16.3 Exemple : problème de planification (interval scheduling)

On a des tâches avec début d_i et fin f_i . On veut maximiser le nombre de tâches compatibles (non chevauchantes).

```

1 def planification(taches):
2     # taches = liste de (debut, fin)
3     taches.sort(key=lambda x: x[1]) # tri par date de fin
4     selection = []
5     dernier_fin = -float('inf')
6     for d, f in taches:
7         if d >= dernier_fin:
8             selection.append((d, f))
9             dernier_fin = f
10    return selection

```

Cet algorithme est optimal : le choix glouton (prendre la tâche qui finit le plus tôt) maximise le nombre de tâches.

16.4 Exemple : rendu de monnaie glouton (systèmes canoniques)

Avec des pièces de valeurs 1, 2, 5, 10, 20, 50, 100, 200, l'algorithme glouton (prendre la plus grande pièce possible) donne le nombre minimal de pièces.

```

1 def rendu_glouton(pieces, somme):
2     pieces.sort(reverse=True)
3     resultat = []
4     for p in pieces:
5         while somme >= p:
6             resultat.append(p)
7             somme -= p
8     return resultat

```

Attention : pour des pièces non canoniques (ex : 1, 3, 4), le glouton peut ne pas être optimal.

16.5 Exemple : sac à dos fractionnaire

Objets avec poids p_i et valeur v_i . On peut emporter une fraction d'objet. Algorithme glouton par rapport au rapport v_i/p_i .

```

1 def sac_dos_fractionnaire(poids, valeurs, capacite):
2     n = len(poids)
3     objets = [(valeurs[i]/poids[i], poids[i], valeurs[i]) for i in
4               range(n)]
5     objets.sort(reverse=True) # tri par valeur massique
6     valeur_totale = 0
7     for ratio, p, v in objets:
8         if capacite >= p:
9             valeur_totale += v
10            capacite -= p
11        else:
12            valeur_totale += ratio * capacite
13            break
14    return valeur_totale

```

Cet algorithme est optimal pour le problème fractionnaire.

17 Bases de données relationnelles et SQL

17.1 Concepts fondamentaux

Une base de données relationnelle est un ensemble de **tables** (relations). Chaque table possède :

- des **attributs** (colonnes) décrivant les propriétés ;
- des **enregistrements** (lignes) correspondant aux données.

Une **clé primaire** est un attribut (ou combinaison) qui identifie de manière unique chaque ligne. Une **clé étrangère** est un attribut qui fait référence à la clé primaire d'une autre table, établissant une relation entre les tables.

17.2 Modèle entité-association (conceptuel)

Avant d'implémenter une base de données, on modélise le domaine sous forme de **entités** (rectangles) et d'**associations** (losanges) avec des **cardinalités** (1-1, 1-n, n-n).

17.3 Modèle relationnel (logique)

Traduction des associations :

- **1-1** : ajouter une clé étrangère dans l'une des tables.
- **1-n** : ajouter une clé étrangère dans la table du côté "n".
- **n-n** : créer une table d'association contenant les deux clés étrangères.

17.4 SQL : Structured Query Language

SQL est le langage standard pour interagir avec les bases de données relationnelles. Les commandes principales sont :

- **DDL** (Data Definition Language) : CREATE, ALTER, DROP.
- **DML** (Data Manipulation Language) : INSERT, UPDATE, DELETE, SELECT.
- **DCL** (Data Control Language) : GRANT, REVOKE.

17.4.1 Création de tables

```
1 CREATE TABLE Departement (  
2     id INTEGER PRIMARY KEY,  
3     nom VARCHAR(50),  
4     description TEXT  
5 );  
6  
7 CREATE TABLE Employee (  
8     id INTEGER PRIMARY KEY,  
9     nom VARCHAR(100),  
10    date_naiss DATE,  
11    annee_emp INTEGER,  
12    ville VARCHAR(50),  
13    id_dep INTEGER,  
14    FOREIGN KEY (id_dep) REFERENCES Departement(id)  
15 );
```

17.4.2 Insertion, mise à jour, suppression

```
1 INSERT INTO Departement (id, nom, description)  
2 VALUES (1, 'Finance', 'Gère les finances');  
3  
4 UPDATE Employee SET ville = 'Paris' WHERE id = 1;  
5  
6 DELETE FROM Employee WHERE id = 2;
```

17.4.3 Interrogation : SELECT

```
1 -- Projection simple  
2 SELECT nom, prenom FROM Etudiant;  
3  
4 -- Sélection avec condition  
5 SELECT * FROM Etudiant WHERE date_naiss > '2000-01-01';  
6  
7 -- Elimination des doublons  
8 SELECT DISTINCT ville FROM Employee;  
9  
10 -- Fonctions d'agrégation  
11 SELECT COUNT(*), AVG(salaire), MIN(salaire), MAX(salaire)  
12 FROM Employee;  
13  
14 -- Regroupement  
15 SELECT ville, COUNT(*) FROM Employee GROUP BY ville;  
16  
17 -- Filtrage après regroupement  
18 SELECT ville, AVG(salaire) FROM Employee  
19 GROUP BY ville
```

```
20 HAVING AVG(salaire) > 3000;
21
22 -- Jointure interne
23 SELECT E.nom, D.nom AS departement
24 FROM Employee E
25 JOIN Departement D ON E.id_dep = D.id;
26
27 -- Jointure externe
28 SELECT E.nom, D.nom
29 FROM Employee E
30 LEFT JOIN Departement D ON E.id_dep = D.id;
31
32 -- Sous-requête
33 SELECT nom FROM Employee
34 WHERE salaire > (SELECT AVG(salaire) FROM Employee);
35
36 -- Opérations ensemblistes
37 SELECT nom FROM Employee WHERE ville = 'Paris'
38 UNION
39 SELECT nom FROM Employee WHERE ville = 'Lyon';
40
41 SELECT nom FROM Employee WHERE ville = 'Paris'
42 INTERSECT
43 SELECT nom FROM Employee WHERE salaire > 3000;
44
45 SELECT nom FROM Employee WHERE ville = 'Paris'
46 EXCEPT
47 SELECT nom FROM Employee WHERE annee_emp < 2010;
```

17.5 Ordre logique d'exécution d'une requête

1. FROM et jointures
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY
7. LIMIT / OFFSET

17.6 Utilisation en Python avec sqlite3

```
1 import sqlite3
2
3 conn = sqlite3.connect('entreprise.db')
4 cur = conn.cursor()
5
6 cur.execute('''
7     CREATE TABLE IF NOT EXISTS Departement (
8         id INTEGER PRIMARY KEY,
9         nom TEXT,
10        description TEXT
11    )
12 ''')
```

```

14 cur.execute("INSERT INTO Departement VALUES (1, 'Finance', 'Gère les
    finances')")
15 conn.commit()
16
17 cur.execute("SELECT * FROM Departement")
18 for ligne in cur.fetchall():
19     print(ligne)
20
21 conn.close()

```

18 Apprentissage automatique

18.1 Introduction

L'apprentissage automatique (Machine Learning) est une branche de l'intelligence artificielle qui permet aux systèmes d'apprendre à partir de données, sans être explicitement programmés. On distingue trois grands paradigmes :

- **Supervisé** : les données sont étiquetées (classification, régression).
- **Non supervisé** : les données ne sont pas étiquetées (clustering, réduction de dimension).
- **Par renforcement** : un agent apprend par essais-erreurs en interagissant avec un environnement.

18.2 Apprentissage supervisé : k plus proches voisins (k-NN)

L'algorithme des k plus proches voisins est un algorithme non paramétrique utilisé pour la classification et la régression.

18.2.1 Principe

Pour classer un nouveau point x :

1. Choisir k (nombre de voisins) et une fonction de distance (souvent la distance euclidienne).
2. Calculer les distances entre x et tous les points de l'ensemble d'entraînement.
3. Sélectionner les k points les plus proches.
4. Affecter à x la classe majoritaire parmi ces k voisins (pour la régression, on prend la moyenne des valeurs).

18.2.2 Distance euclidienne

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

```

1 def euclidean_distance(x, y):
2     """Calcule la distance euclidienne entre deux points de même
    dimension."""
3     distance = 0.0
4     for i in range(len(x)):
5         distance += (x[i] - y[i])**2
6     return distance**0.5

```

Avec NumPy :

```

1 import numpy as np
2 def euclidean_distance_np(x, y):
3     return np.sqrt(np.sum((np.array(x)-np.array(y))**2))

```


18.2.3 Algorithme k-NN complet

```

1 from collections import Counter
2
3 def knn(X_train, y_train, x_test, k=3):
4     distances = []
5     for i in range(len(X_train)):
6         d = euclidean_distance(x_test, X_train[i])
7         distances.append((d, y_train[i]))
8     distances.sort(key=lambda t: t[0])
9     k_nearest = [label for _, label in distances[:k]]
10    return Counter(k_nearest).most_common(1)[0][0]

```

18.2.4 Choix de k et normalisation

- k trop petit : sensible au bruit (surapprentissage).
- k trop grand : lissage excessif.
- Normaliser les données (Z-score ou mise à l'échelle) pour éviter qu'une variable domine les distances.

18.3 Apprentissage non supervisé : k-moyennes (k-means)

L'algorithme k-means partitionne les données en k groupes (clusters) en minimisant la variance intra-cluster.

18.3.1 Algorithme

1. Initialiser k centroïdes (souvent aléatoirement parmi les points).
2. Affecter chaque point au centroïde le plus proche.
3. Recalculer chaque centroïde comme la moyenne des points de son cluster.
4. Répéter 2-3 jusqu'à convergence (les centroïdes ne bougent plus).

```

1 import numpy as np
2
3 def kmeans(X, k, max_iter=100):
4     # Initialisation aléatoire
5     centroids = X[np.random.choice(len(X), k, replace=False)]
6     for _ in range(max_iter):
7         # Affectation
8         distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)
9         labels = np.argmin(distances, axis=1)
10        # Mise à jour
11        new_centroids = np.array([X[labels == i].mean(axis=0) for i in
12                                   range(k)])
13        if np.allclose(centroids, new_centroids):
14            break
15        centroids = new_centroids
16    return labels, centroids

```

18.3.2 Choix du nombre de clusters

- Méthode du coude (elbow) : tracer l'inertie intra-cluster en fonction de k et choisir le point où la courbe s'infléchit.
- Indice de silhouette.

18.4 Évaluation des performances en classification

18.4.1 Matrice de confusion

	Prédit positif	Prédit négatif
Réel positif	VP (vrai positif)	FN (faux négatif)
Réel négatif	FP (faux positif)	VN (vrai négatif)

Métriques courantes :

- Précision = $\frac{VP}{VP+FP}$
- Rappel = $\frac{VP}{VP+FN}$
- F1-score = $2 \times \frac{\text{précision} \times \text{rappel}}{\text{précision} + \text{rappel}}$
- Exactitude = $\frac{VP+VN}{VP+VN+FP+FN}$

19 Théorie des jeux

19.1 Définitions

On étudie les jeux à deux joueurs, à information parfaite, déterministes, à tour de rôle.

- **Jeu impartial** : les deux joueurs ont les mêmes coups possibles dans une position donnée.
- **Jeu d'accessibilité** : modélisé par un graphe orienté acyclique (DAG). Le joueur qui ne peut plus jouer perd (convention normale) ou gagne (convention misère).
- **Position gagnante (N-position)** : le joueur dont c'est le tour peut forcer la victoire.
- **Position perdante (P-position)** : le joueur dont c'est le tour perd si l'adversaire joue parfaitement.

Dans un jeu impartial, une position est gagnante s'il existe un coup vers une position perdante ; elle est perdante si tous les coups mènent à des positions gagnantes.

19.2 Attracteurs

Pour un jeu d'accessibilité avec condition de gain, on construit l'ensemble des positions gagnantes par induction.

```

1 def attracteur(graphe, but, joueur):
2     # graphe : liste d'adjacence
3     # but : ensemble des sommets gagnants pour le joueur
4     # joueur : 0 ou 1 (contrôle des sommets)
5     n = len(graphe)
6     gagnant = set(but)
7     change = True
8     while change:
9         change = False
10        for u in range(n):
11            if u in gagnant:
12                continue
13            if u % 2 == joueur: # sommet contrôlé par le joueur
14                if any(v in gagnant for v in graphe[u]):
15                    gagnant.add(u)
16                    change = True
17            else: # sommet contrôlé par l'adversaire
18                if all(v in gagnant for v in graphe[u]):
19                    gagnant.add(u)
20                    change = True
21    return gagnant

```

19.3 Jeu de Nim et nombre de Grundy

Le jeu de Nim se joue avec plusieurs tas d'allumettes. À chaque tour, un joueur retire un nombre quelconque d'allumettes d'un seul tas. Celui qui prend la dernière gagne.

- Le **nimbre** (ou somme de Nim) est le XOR (ou exclusif) des tailles des tas.
- Une position est perdante (P-position) si et seulement si le nimbre est nul.

```

1 def nim_position(tas):
2     """Retourne True si position gagnante."""
3     nim_sum = 0
4     for t in tas:
5         nim_sum ^= t
6     return nim_sum != 0

```

Le **nombre de Grundy** généralise cette notion à tout jeu impartial. Pour un sommet x ,

$$g(x) = \text{mex}(\{g(y) \mid (x, y) \in A\})$$

où mex (minimum excluded) est le plus petit entier naturel absent de l'ensemble. Une position est perdante ssi $g(x) = 0$.

19.4 Algorithme Min-Max (pour jeux partisans)

Pour les jeux avec évaluation possible des positions (ex : échecs), on explore l'arbre de jeu en alternant les niveaux :

- **Joueur MAX** : choisit le coup maximisant son gain.
- **Joueur MIN** : choisit le coup minimisant le gain de MAX.

```

1 def minimax(noeud, profondeur, maximisant, heuristique):
2     if profondeur == 0 or est_terminal(noeud):
3         return heuristique(noeud)
4     if maximisant:
5         meilleur = -float('inf')
6         for fils in successeurs(noeud):
7             val = minimax(fils, profondeur-1, False, heuristique)
8             meilleur = max(meilleur, val)
9         return meilleur
10    else:
11        meilleur = float('inf')
12        for fils in successeurs(noeud):
13            val = minimax(fils, profondeur-1, True, heuristique)
14            meilleur = min(meilleur, val)
15        return meilleur

```

19.5 Élagage alpha-bêta

Optimisation de Min-Max qui évite d'explorer des branches inutiles. On maintient deux bornes :

- α : meilleur score garanti pour MAX.
- β : meilleur score garanti pour MIN.

Si $\alpha \geq \beta$, on peut couper.

```

1 def alphabeta(noeud, profondeur, alpha, beta, maximisant, heuristique):
2     if profondeur == 0 or est_terminal(noeud):
3         return heuristique(noeud)
4     if maximisant:
5         for fils in successeurs(noeud):

```

```

6         alpha = max(alpha, alphabeta(fils, profondeur-1, alpha, beta,
7             False, heuristique))
8         if alpha >= beta:
9             break
10        return alpha
11    else:
12        for fils in successeurs(noeud):
13            beta = min(beta, alphabeta(fils, profondeur-1, alpha, beta,
14                True, heuristique))
15            if beta <= alpha:
16                break
17        return beta

```

19.6 Heuristique

Pour des jeux complexes, on ne peut pas explorer tout l'arbre. On utilise une fonction heuristique $h(n)$ qui estime la qualité de la position n pour MAX (sans explorer plus loin).

20 Algorithmes de graphes avancés

20.1 Algorithme de Bellman-Ford (plus courts chemins avec poids négatifs)

Contrairement à Dijkstra, Bellman-Ford autorise des poids négatifs et détecte les cycles absorbants (cycles de poids total négatif).

Principe : on relaxe toutes les arêtes $|V| - 1$ fois. Complexité $O(V \cdot E)$.

```

1 def bellman_ford(graphe, source):
2     n = len(graphe)
3     dist = [float('inf')] * n
4     dist[source] = 0
5     prev = [None] * n
6     for _ in range(n-1):
7         mise_a_jour = False
8         for u in range(n):
9             if dist[u] == float('inf'):
10                continue
11            for v, w in graphe[u]:
12                if dist[u] + w < dist[v]:
13                    dist[v] = dist[u] + w
14                    prev[v] = u
15                    mise_a_jour = True
16        if not mise_a_jour:
17            break
18    # Détection de cycle négatif
19    for u in range(n):
20        if dist[u] == float('inf'):
21            continue
22        for v, w in graphe[u]:
23            if dist[u] + w < dist[v]:
24                raise ValueError("Cycle absorbant détecté")
25    return dist, prev

```

20.2 Algorithme de Floyd-Warshall (tous les couples)

Calcule les plus courts chemins entre toutes les paires. Accepte les poids négatifs (pas de cycle négatif). Complexité $O(V^3)$.

```

1 def floyd_warshall(M):
2     n = len(M)
3     dist = [[M[i][j] for j in range(n)] for i in range(n)]
4     for k in range(n):
5         for i in range(n):
6             for j in range(n):
7                 if dist[i][k] + dist[k][j] < dist[i][j]:
8                     dist[i][j] = dist[i][k] + dist[k][j]
9     return dist

```

20.3 Algorithme de Johnson (tous les couples, poids négatifs)

Pour graphes orientés pondérés avec poids négatifs mais sans cycle négatif. Combine Bellman-Ford et Dijkstra. Complexité $O(VE + V^2 \log V)$. Étapes :

1. Ajouter un sommet s connecté à tous les sommets (poids 0).
2. Bellman-Ford depuis $s \rightarrow$ potentiels $h(v)$.
3. Rééchelonner les poids : $w'(u, v) = w(u, v) + h(u) - h(v)$ (tous positifs).
4. Dijkstra depuis chaque sommet sur le graphe rééchelonné.
5. Distances réelles : $d(u, v) = d'(u, v) - h(u) + h(v)$.

20.4 Arbres couvrants de poids minimal

20.4.1 Algorithme de Prim

Construit l'arbre couvrant minimal en ajoutant le sommet le plus proche. Utilise une file de priorité. Complexité $O((V + E) \log V)$.

```

1 import heapq
2
3 def prim(graphe, depart=0):
4     n = len(graphe)
5     visites = [False] * n
6     tas = [(0, depart, -1)]
7     acpm = []
8     total = 0
9     while tas:
10         poids, u, parent = heapq.heappop(tas)
11         if visites[u]:
12             continue
13         visites[u] = True
14         if parent != -1:
15             acpm.append((parent, u, poids))
16             total += poids
17         for v, w in graphe[u]:
18             if not visites[v]:
19                 heapq.heappush(tas, (w, v, u))
20     return acpm, total

```

20.4.2 Algorithme de Kruskal

Trie les arêtes par poids croissant et les ajoute si elles ne créent pas de cycle (union-find). Complexité $O(E \log E)$.

```

1 def trouver(pere, x):
2     if pere[x] != x:
3         pere[x] = trouver(pere, pere[x])

```

```

4     return pere[x]
5
6 def union(pere, rang, x, y):
7     xr, yr = trouver(pere, x), trouver(pere, y)
8     if xr == yr:
9         return False
10    if rang[xr] < rang[yr]:
11        pere[xr] = yr
12    elif rang[xr] > rang[yr]:
13        pere[yr] = xr
14    else:
15        pere[yr] = xr
16        rang[xr] += 1
17    return True
18
19 def kruskal(graphe):
20     aretes = []
21     n = len(graphe)
22     for u in range(n):
23         for v, w in graphe[u]:
24             if u < v:
25                 aretes.append((w, u, v))
26     aretes.sort()
27     pere = list(range(n))
28     rang = [0]*n
29     acpm = []
30     total = 0
31     for w, u, v in aretes:
32         if union(pere, rang, u, v):
33             acpm.append((u, v, w))
34             total += w
35     return acpm, total

```

20.5 Algorithme A* (recherche de chemin avec heuristique)

A* combine le coût réel $g(n)$ et une heuristique $h(n)$. Priorité : $f(n) = g(n) + h(n)$.

```

1 import heapq
2
3 def a_star(graphe, depart, but, heuristique):
4     n = len(graphe)
5     dist = [float('inf')] * n
6     dist[depart] = 0
7     prev = [None] * n
8     pq = [(heuristique(depart), 0, depart)]
9     while pq:
10         _, cost, u = heapq.heappop(pq)
11         if u == but:
12             break
13         if cost > dist[u]:
14             continue
15         for v, w in graphe[u]:
16             nd = cost + w
17             if nd < dist[v]:
18                 dist[v] = nd
19                 prev[v] = u

```

```

20         heapq.heappush(pq, (nd + heuristique(v), nd, v))
21     # Reconstruction du chemin
22     chemin = []
23     u = but
24     while u is not None:
25         chemin.append(u)
26         u = prev[u]
27     return chemin[::-1]

```

Une heuristique est **admissible** si elle ne surestime jamais le coût réel. Elle est **monotone** si $h(n) \leq w(n, m) + h(m)$. A* avec une heuristique admissible garantit l'optimalité.

20.6 Ordre topologique (graphe orienté acyclique)

Tri des sommets tel que pour tout arc (u, v) , u apparaît avant v .

```

1 def tri_topologique(G):
2     n = len(G)
3     visites = [False]*n
4     ordre = []
5     def dfs(u):
6         visites[u] = True
7         for v in G[u]:
8             if not visites[v]:
9                 dfs(v)
10        ordre.append(u)
11    for i in range(n):
12        if not visites[i]:
13            dfs(i)
14    return ordre[::-1]

```

20.7 Composantes fortement connexes (Kosaraju)

```

1 def kosaraju(G):
2     n = len(G)
3     visites = [False]*n
4     ordre = []
5     def dfs1(u):
6         visites[u] = True
7         for v in G[u]:
8             if not visites[v]:
9                 dfs1(v)
10        ordre.append(u)
11    for i in range(n):
12        if not visites[i]:
13            dfs1(i)
14    # Graphe transposé
15    GT = [[] for _ in range(n)]
16    for u in range(n):
17        for v in G[u]:
18            GT[v].append(u)
19    visites = [False]*n
20    cfc = []
21    def dfs2(u, comp):
22        visites[u] = True
23        comp.append(u)
24        for v in GT[u]:

```

```

25         if not visites[v]:
26             dfs2(v, comp)
27     while ordre:
28         u = ordre.pop()
29         if not visites[u]:
30             comp = []
31             dfs2(u, comp)
32             cfc.append(comp)
33     return cfc

```

21 Tables de hachage et recherche de motif

21.1 Tables de hachage

Une table de hachage (ou dictionnaire) associe des clés à des valeurs en $O(1)$ amorti. Elle repose sur une **fonction de hachage** h .

21.1.1 Fonction de hachage

Doit être rapide, uniforme et déterministe.

- Pour des entiers : $h(k) = k \bmod m$ (m premier).
- Pour des chaînes : $h(s) = \left(\sum_{i=0}^{n-1} \text{ord}(s[i]) \cdot b^{n-1-i} \right) \bmod m$.

21.1.2 Gestion des collisions

Chaînage : chaque case pointe vers une liste de couples (clé, valeur).

```

1 class HashMapChaining:
2     def __init__(self, size=100):
3         self.size = size
4         self.table = [[] for _ in range(size)]
5     def _hash(self, key):
6         return hash(key) % self.size
7     def put(self, key, value):
8         idx = self._hash(key)
9         for i, (k, v) in enumerate(self.table[idx]):
10             if k == key:
11                 self.table[idx][i] = (key, value)
12                 return
13         self.table[idx].append((key, value))
14     def get(self, key):
15         idx = self._hash(key)
16         for k, v in self.table[idx]:
17             if k == key:
18                 return v
19         raise KeyError(key)

```

Adressage ouvert (sondage linéaire) : si la case est occupée, on essaie la suivante.

```

1 class HashMapOpen:
2     def __init__(self, size=100):
3         self.size = size
4         self.table = [None] * size
5         self.deleted = object()
6     def _hash(self, key):
7         return hash(key) % self.size
8     def put(self, key, value):
9         idx = self._hash(key)

```



```

10     while self.table[idx] not in (None, self.deleted):
11         if self.table[idx][0] == key:
12             self.table[idx] = (key, value)
13             return
14         idx = (idx + 1) % self.size
15     self.table[idx] = (key, value)
16 def get(self, key):
17     idx = self._hash(key)
18     start = idx
19     while self.table[idx] is not None:
20         if self.table[idx] is not self.deleted and self.table[idx][0]
21             == key:
22             return self.table[idx][1]
23         idx = (idx + 1) % self.size
24         if idx == start:
25             break
26     raise KeyError(key)

```

21.2 Recherche de motif dans un texte

21.2.1 Algorithme naïf

Complexité $O(nm)$.

21.2.2 Algorithme de Rabin-Karp

Utilise un hachage roulant. Complexité moyenne $O(n + m)$.

```

1 def rabin_karp(texte, motif):
2     n, m = len(texte), len(motif)
3     if m > n: return []
4     b = 256; q = 101
5     h = pow(b, m-1, q)
6     hash_motif = 0; hash_fenetre = 0
7     for i in range(m):
8         hash_motif = (hash_motif * b + ord(motif[i])) % q
9         hash_fenetre = (hash_fenetre * b + ord(texte[i])) % q
10    resultats = []
11    for i in range(n - m + 1):
12        if hash_motif == hash_fenetre and texte[i:i+m] == motif:
13            resultats.append(i)
14        if i < n - m:
15            hash_fenetre = ((hash_fenetre - ord(texte[i]) * h) * b +
16                            ord(texte[i+m])) % q
17    return resultats

```

21.2.3 Algorithme de Knuth-Morris-Pratt (KMP)

Évite les retours en arrière grâce à un tableau de préfixes. Complexité $O(n + m)$.

```

1 def construire_tableau_prefixe(motif):
2     m = len(motif)
3     pi = [0] * m
4     k = 0
5     for q in range(1, m):
6         while k > 0 and motif[k] != motif[q]:
7             k = pi[k-1]
8         if motif[k] == motif[q]:
9             k += 1

```

```

10     pi[q] = k
11     return pi
12
13 def kmp(texte, motif):
14     n, m = len(texte), len(motif)
15     if m == 0: return []
16     pi = construire_tableau_prefixe(motif)
17     q = 0
18     resultats = []
19     for i in range(n):
20         while q > 0 and motif[q] != texte[i]:
21             q = pi[q-1]
22         if motif[q] == texte[i]:
23             q += 1
24         if q == m:
25             resultats.append(i - m + 1)
26             q = pi[q-1]
27     return resultats

```

22 Bonnes pratiques de programmation

22.1 Documentation et lisibilité

Un code doit être compréhensible par d'autres développeurs (et par soi-même dans le futur).

- Utiliser des noms de variables explicites (éviter `a`, `b`, `tmp`).
- Suivre les conventions PEP 8 : noms en `snake_case` pour les variables/fonctions, `CamelCase` pour les classes.
- Ajouter une docstring pour chaque fonction (description, paramètres, retour).

```

1 def factorielle(n: int) -> int:
2     """Calcule la factorielle de n (n entier >= 0)."""
3     if n <= 1:
4         return 1
5     return n * factorielle(n-1)

```

22.2 Assertions et programmation défensive

Les **assertions** permettent de vérifier des préconditions et des invariants pendant le développement.

```

1 def division(a, b):
2     assert b != 0, "Division par zéro"
3     return a / b

```

En production, les assertions peuvent être désactivées (option `-O`).

22.3 Gestion des exceptions

Utiliser des exceptions pour signaler des erreurs, plutôt que des codes de retour.

```

1 def lire_fichier(nom):
2     try:
3         with open(nom, 'r') as f:
4             return f.read()
5     except FileNotFoundError:
6         print(f"Fichier {nom} introuvable")
7         return None

```

22.4 Modularité et réutilisabilité

Découper le code en petites fonctions avec une seule responsabilité. Regrouper les fonctions liées dans des modules. Utiliser `if __name__ == "__main__":` pour rendre un module exécutable.

22.5 Tests unitaires

Écrire des tests automatisés avec `unittest` ou `pytest` pour valider le comportement du code.

```

1 import unittest
2
3 class TestMath(unittest.TestCase):
4     def test_factorielle(self):
5         self.assertEqual(factorielle(0), 1)
6         self.assertEqual(factorielle(5), 120)
7
8 if __name__ == "__main__":
9     unittest.main()

```

22.6 Contrôle de version

Utiliser Git pour suivre l'évolution du code. Commiter régulièrement avec des messages clairs.

23 Preuves de correction et terminaison

23.1 Terminaison d'une boucle

Pour prouver qu'une boucle `while` termine, on exhibe un **variant** : une suite d'entiers naturels strictement décroissante à chaque itération. Exemple : calcul de la division euclidienne.

```

1 def division(a, b):
2     r = a
3     q = 0
4     # variant : r
5     while r >= b:
6         r -= b
7         q += 1
8     return q, r

```

Le variant r est positif et diminue strictement à chaque tour $\Rightarrow$ nombre d'itérations fini.

23.2 Correction d'une boucle : invariant

Un **invariant** est une propriété vraie avant la boucle et conservée par chaque itération. À la sortie, l'invariant et la négation de la condition de boucle donnent le résultat voulu.

Exemple : recherche dichotomique.

```

1 def recherche_dichotomique(t, x):
2     g, d = 0, len(t)-1
3     # invariant : x est dans t[g:d+1] s'il existe
4     while g <= d:
5         m = (g+d)//2
6         if t[m] == x:
7             return m
8         elif t[m] < x:
9             g = m+1
10        else:
11            d = m-1
12    return -1

```

23.3 Correction récursive

Pour une fonction récursive, on prouve par récurrence sur la taille des données.

```

1 def fibo(n):
2     if n <= 1:
3         return 1
4     return fibo(n-1) + fibo(n-2)

```

Hypothèse : pour tout $k < n$, `fibo(k)` calcule correctement F_k . Vérification pour $n = 0, 1$ (cas de base). Pour $n > 1$, la somme des deux appels récursifs donne $F_{n-1} + F_{n-2} = F_n$.

23.4 Complexité et preuve

On combine terminaison et correction pour obtenir une **correction totale** : l'algorithme termine et donne le bon résultat.

24 Récursivité avancée

24.1 Récursivité terminale

Une fonction est dite à **récursivité terminale** si l'appel récursif est la dernière opération. Cela permet à certains compilateurs d'optimiser (élimination de la récursion) pour éviter la surcharge de pile.

```

1 def factorielle_term(n, acc=1):
2     if n <= 1:
3         return acc
4     return factorielle_term(n-1, n*acc)    # appel terminal

```

24.2 Mémoïsation (caching)

Pour éviter de recalculer plusieurs fois les mêmes sous-problèmes, on stocke les résultats déjà calculés.

```

1 from functools import lru_cache
2
3 @lru_cache(maxsize=None)
4 def fibo_cache(n):
5     if n <= 1:
6         return 1
7     return fibo_cache(n-1) + fibo_cache(n-2)

```

Le décorateur `@lru_cache` implémente automatiquement la mémoïsation.

24.3 Récursivité mutuelle

Deux fonctions s'appellent l'une l'autre. Exemple : déterminer si un nombre est pair ou impair.

```

1 def est_pair(n):
2     if n == 0:
3         return True
4     return est_impair(n-1)
5
6 def est_impair(n):
7     if n == 0:
8         return False
9     return est_pair(n-1)

```

24.4 Récursivité sur des structures arborescentes

Parcours récursif des arbres binaires (vu au chapitre 10). La profondeur de récursion est égale à la hauteur de l'arbre.

24.5 Dépassement de pile et profondeur limite

Python limite la profondeur de récursion (par défaut 1000). On peut la modifier avec `sys.setrecursionlimit()`. Pour traiter des données de grande taille, on préfère une version itérative ou une récursion terminale optimisée.